

[gobble]

Natural Language Access to Inorganic Materials Relational Databases: Design Principles and Validation via Schema-Graph-Constrained Text-to-SQL

Satoshi Minamoto^{1*}

^{1*}Research Center for Materials Nanoarchitectonics, National Institute
for Materials Science (NIMS), 1-2-1 Sengen, Tsukuba, 305-0047, Ibaraki,
Japan.

Corresponding author(s). E-mail(s): minamoto.satoshi@nims.go.jp;

Abstract

Inorganic materials databases require SQL knowledge, creating a significant barrier for materials researchers. We present the design principles for a Text-to-SQL pipeline that enables natural language access to inorganic materials relational databases (RDBs) and demonstrate its effectiveness. The proposed method comprises six components: few-shot example retrieval, LLM-based SQL generation, Steiner-tree JOIN constraints derived from the foreign-key graph, a materials domain dictionary, AST-based SQL safety verification, and an LLM reranker. Using a 31-table RDB centered on L1₂-type intermetallic compounds, we conducted a 7-condition ablation study with 100 evaluation queries (5 independent runs, 3,500 evaluations in total). Few-shot examples and the materials domain dictionary contributed -7.4 pp ($p < \mathbf{0.001}$) and -7.3 pp ($p < \mathbf{0.001}$), respectively, demonstrating statistically significant effects. Multi-axis evaluation achieved execution accuracy of 84.7%, syntax validity of 100%, and JOIN match rate of 93.2%. All results are reported as mean $\pm$ standard deviation over 5 independent runs with Wilcoxon signed-rank test significance. Furthermore, zero-adaptation transfer evaluations across four schema variants—prototype expansion (A) 90.0%, OQMD-flavored rename (B) 90.5%, randomized obfuscated names (C) 90.0%, and a real Materials Project-flavored schema (D) 100.0%—demonstrated that the pipeline generalizes schema-independently. External validation with 100 independently designed queries (76.9%) and evaluation of 15 CTE multi-step calculation patterns quantify the external validity and applicability limits of the design principles.

Keywords: Text-to-SQL, inorganic materials database, natural language interface, schema graph, domain-specific dictionary, materials informatics

1 Introduction

1.1 Barriers to Data Exploration in Inorganic Materials Research

Inorganic materials research is transitioning toward data-driven materials discovery driven by advances in high-throughput first-principles calculations. Databases such as OQMD [1, 2], Materials Project [3], AFLOW [4], and NOMAD [5] store tens to hundreds of thousands of computational records covering composition, crystal structure, electronic states, and thermodynamic stability.

These data are typically stored in normalized relational databases (RDBs) designed for flexible cross-table queries via SQL. However, utilizing RDBs requires understanding table structures, foreign-key (FK) relationships, and SQL syntax—a significant barrier for materials researchers who may know relevant data exists but cannot access it. Consequently, valuable data remains underutilized because researchers cannot query it in natural language.

1.2 Significance of Natural Language Database Interfaces

The core of this barrier is that the technical requirement of SQL writing interposes between domain knowledge and data. A natural language interface is not merely a convenience tool that substitutes for SQL, but a component of research infrastructure that can fundamentally reduce data access barriers and thereby accelerate research.

In inorganic materials RDBs, normalization often results in dozens of tables, and constructing queries involving multi-table JOINS is difficult even for experienced SQL users. If an appropriate SQL can be automatically generated from natural language, materials researchers could perform cross-table data exploration without being aware of the schema structure.

While building a system that fully automatically tracks schema changes and table additions is currently challenging, adopting a design that relies on FK metadata makes adaptation to schema evolution relatively straightforward.

1.3 Text-to-SQL Technology and the Current State in Materials Science

Text-to-SQL technology has developed around benchmarks such as Spider [6] and BIRD [7]. With recent advances in large language models (LLMs), DIN-SQL [8] proposed a pipeline approach separating schema linking, classification, and generation; DAIL-SQL [9] focused on few-shot example selection strategies. RAT-SQL [10] introduced relation-aware encoding, and RESDSQL [11] introduced schema link decoupling. SchemaGraphSQL [12] is closest to our work in that it uses path finding on the FK graph.

However, these methods are evaluated on general-purpose benchmarks and have not been applied to domain-specific RDBs—particularly inorganic materials

databases. Research on natural language interfaces for materials databases remains limited. MatSciBERT [13] and MatBERT [14] are language models for materials science text but have not been applied directly to Text-to-SQL tasks. MKNA [15] and OptiMat [16] provide API-based access but do not perform SQL generation with JOIN control on normalized RDBs. SuperCon [17] and PoLyInfo [18] take RDF/SPARQL-based ontology approaches with strong semantic reasoning, but require data conversion for application to existing RDB infrastructure.

1.4 Objectives and Contributions

This study aims to present design principles for enabling natural language access to inorganic materials RDBs and to validate their effectiveness. We demonstrate a methodology that allows materials researchers to explore data without being aware of SQL or schema structure, using OQMD as a representative example to examine the extent to which natural language queries can be handled.

The three domain-specific challenges addressed in this study are:

1. Domain terminology ambiguity: Crystal structure names and property terms such as $L1_2$, γ' phase, and B2 are insufficiently represented in general-purpose LLM training data, making accurate conversion to SQL conditions difficult.
2. Multi-table JOIN complexity: Normalized materials RDBs require JOINS across dozens of tables, creating a high risk of JOIN hallucination by LLMs.
3. Absence of evaluation data: No standard evaluation benchmark exists for Text-to-SQL on inorganic materials RDBs.

Our contributions are threefold:

- (i) Design principles for materials-RDB Text-to-SQL: We propose a pipeline integrating Steiner-tree JOIN constraints on FK graphs, a materials domain dictionary, and a hybrid reranker. This fills the gap between general-purpose Text-to-SQL methods (DIN-SQL, DAIL-SQL, SchemaGraphSQL, etc.) that lack materials-specific term translation, and materials NL access systems (MKNA, OptiMat, etc.) that do not target RDB/SQL.
- (ii) Multi-axis quantitative evaluation: Beyond the 7-condition $\times$ 100-query $\times$ 5-run statistical ablation (Wilcoxon signed-rank test) on a 31-table $L1_2$ RDB, we provide few-shot k sensitivity, dictionary size sensitivity, LLM model comparison, and multi-axis metrics (EM / SELECT column precision / JOIN match rate).
- (iii) Quantitative effect of domain knowledge: We demonstrate that the materials dictionary (-7.3 pp, $p < 0.001$) and few-shot examples (-7.4 pp, $p < 0.001$) have statistically significant contributions, validating the necessity of domain-specific modules.
- (iv) Demonstration of schema generalization: Zero-adaptation transfer tests across four schema variants—A prototype expansion (90.0%), B OQMD-flavored rename (90.5%), C randomized obfuscation (90.0%), and D Materials Project-flavored schema (100.0%)—demonstrate that the design principles based on FK introspection do not depend on a specific schema (Section 10.13).

2 Proposed Method

2.1 Pipeline Overview

To enable natural language access to inorganic materials RDBs, we designed the pipeline shown in Fig. 1. It receives a natural language query (Japanese) as input, performs materials domain term analysis, automatic schema traversal, SQL generation and verification, and returns the execution results. Sections 2.2–2.10 describe the overall processing flow, followed by Sections 2.6–2.7 addressing challenges specific to materials RDBs.

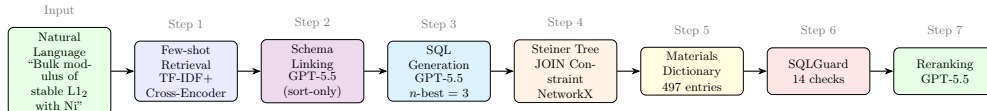


Fig. 1: Overview of the proposed pipeline. From a natural language query, few-shot retrieval (Step 1) is followed by GPT-5.5 schema linking (Step 2) and SQL generation (Step 3). Steiner-tree approximation (Step 4) extracts the minimal subgraph from 31 tables. The materials dictionary (Step 5) maps domain terms to SQL conditions, SQLGuard (Step 6) verifies safety, and the reranker (Step 7) selects the best candidate.

2.2 Few-shot Example Retrieval

Few-shot examples similar to the input query are selected in two stages. First, TF-IDF [19] narrows the candidates to $2k$ items, then a Cross-Encoder (ms-marco-MiniLM-L-6-v2) selects the top k . The Cross-Encoder runs locally with processing time under 50 ms.

In the ablation study, removing few-shot examples decreased overall accuracy from 84.7% to 77.3% (-7.4 pp, $p < 0.001$; Table 6). This decrease was observed for Medium difficulty and above (no decrease for Easy), with -9.2 pp for Medium (Table 7). The LLM acquires SQL generation patterns (table names, JOIN ordering, column name usage) via in-context learning [20]; without few-shot examples, the LLM must infer SQL solely from schema information.

2.3 Schema Linking

GPT-5.5 scores and sorts tables relevant to the input query (sort-only approach). No tables are excluded; only the context priority order for SQL generation is modified.

2.4 SQL Generation and n -best Candidates

GPT-5.5 generates n -best = 3 SQL candidates. In the ablation study, switching to n -best = 1 showed no statistically significant accuracy difference (-0.6 pp, $p = 0.069$), but reduced average latency from 31.0 s to 10.9 s (Table 6).

2.5 Steiner Tree JOIN Constraint

The FK relationships among 31 tables are represented as an undirected graph using NetworkX [21]. FK relationships are automatically obtained at runtime via introspection of PostgreSQL metadata. For the set of tables required by a query, the minimum JOIN subtree is determined via Steiner tree approximation as shown in Algorithm 1.

Algorithm 1 Minimum covering JOIN subgraph (Steiner tree approximation).

Require: FK relationship graph $G = (V, E)$, required table set R

Ensure: JOIN path list P

```

1:  $P \leftarrow \emptyset$ ; initialize Union-Find  $U$ 
2: for all  $(s, t) \in \binom{R}{2}$  do
3:   if Find( $s$ ) = Find( $t$ ) then
4:     continue
5:   end if
6:    $p \leftarrow \text{ShortestPath}(G, s, t)$  ▷ BFS on FK graph
7:   if  $p \neq \emptyset$  then
8:      $P \leftarrow P \cup \{p\}$ 
9:     for all adjacent node pairs  $(u, v) \in p$  do
10:      Union( $u, v$ )
11:    end for
12:   end if
13: end for
14: return  $P$ 

```

For $|V| = n$ nodes and $|R| = k$ required tables, the algorithm performs $O(k^2)$ BFS traversals. At the 31-table scale ($n = 31, k \leq 5$), this completes in milliseconds.

In the ablation study, removing the Steiner tree constraint showed no statistically significant accuracy difference (+0.0 pp, $p = 1.000$; Table 6). Within the validated scope, the current 31-table scale has most queries completed using 5 core tables (`material_entry`, `structure`, `element`, `composition`, `calculated_property`), limiting the observable effect. The contribution is expected to become apparent as the number of tables increases.

2.5.1 Graph Construction from PostgreSQL Metadata

FK relationships are dynamically extracted at runtime from PostgreSQL’s `information_schema`:

Listing 1: FK relationship extraction

```

SELECT tc.table_name, kcu.column_name,
       ccu.table_name, ccu.column_name
FROM   information_schema.table_constraints tc
JOIN   information_schema.key_column_usage kcu
ON     tc.constraint_name = kcu.constraint_name
JOIN   information_schema.constraint_column_usage ccu
ON     ccu.constraint_name = tc.constraint_name
WHERE  tc.constraint_type = 'FOREIGN_KEY';

```

The result is stored as an undirected graph $G = (V, E)$ in NetworkX [21]. Each edge retains the FK direction as an attribute, used during JOIN clause generation. Since FK metadata is read at runtime, the system can track schema changes.

2.5.2 Multi-element AND Queries: EXISTS Subqueries

A materials-specific challenge arising from composition table normalization is queries such as “compounds containing both Ni and Al.” The naive approach `WHERE c.element = 'Ni' AND c.element = 'Al'` always returns 0 rows because no single row satisfies both conditions. The Schema Graph engine detects when multiple elements are specified and automatically generates EXISTS subqueries:

Listing 2: Multi-element AND

```
SELECT DISTINCT m.entry_id, m.formula
FROM material_entry m
WHERE EXISTS (SELECT 1 FROM composition
  WHERE entry_id = m.entry_id AND element = 'Ni')
AND EXISTS (SELECT 1 FROM composition
  WHERE entry_id = m.entry_id AND element = 'Al')
LIMIT 10000;
```

The AND/OR intent is classified by the condition extractor from conjunctions and particles; for OR indicators, an IN clause is used.

2.5.3 SQL Generation from JOIN Paths

Each path in P is converted to a SQL JOIN clause. Table aliases are assigned from a fixed dictionary for all 31 tables (`material_entry`→`m`, `composition`→`c`, etc.). When the same table appears multiple times, numeric suffixes generate unique aliases (`c2`, `c3`).

2.6 Condition Extractor

The Condition Extractor converts natural language queries into structured condition dictionaries using a YAML-based bilingual (Japanese–English) terminology dictionary (`material_terms.yaml`).

2.6.1 Dictionary Structure

The dictionary covers four categories: (1) Prototype aliases (e.g., `L12` ↔ [`L12`, `Cu3Au-type`, `γ`]), (2) Element mappings (`Fe` ↔ [`iron`]), (3) Stability terms (“stable” → `energy_above_hull ≤ 0.001 eV/atom`; “metastable” → `0.001 < Ehull ≤ 0.05`), (4) Property terms (“lattice constant” → `structure.lattice_a`, etc.).

Unicode subscript characters (U+2080–U+2089) are normalized to ASCII digits before matching. Element symbol recognition uses word-boundary regular expressions to distinguish standalone “Fe” from the substring in “FeCoNi”.

2.6.2 Numeric Condition Parser

The parser recognizes numeric comparison expressions via five patterns (Table 1). Property-to-column mapping is managed by an internal dictionary

Table 1: Numeric condition parser patterns.

Pattern type	Input example	Output
Symbol comparison	<code>band_gap > 1.0</code>	<code>WHERE band_gap > 1.0</code>
Japanese postfix	formation energy ≥ 0.5	<code>WHERE ... >= 0.5</code>
Japanese comparative	lattice const. ≥ 3.0	<code>WHERE lattice_a > 3.0</code>
Sign determination	negative formation energy	<code>WHERE ... < 0</code>
Range specification	band gap 0.5–2.0 eV	<code>WHERE ... BETWEEN</code>

(`_PROPERTY_COLUMN_MAP`). For floating-point comparisons, expressions such as “near” or “comparable” invoke property-specific tolerances (lattice constant: ± 0.03 Å; E_{hull} : ± 0.001 eV/atom).

2.6.3 Chemical Formula Parser and Ambiguity Handling

Chemical formula tokens are classified into three types: (1) `exact_formula` (`Ni3Al` $\rightarrow$ `formula = 'Ni3Al'`), (2) `formula_or_reduced_formula` (“B2 entries of NiAl” $\rightarrow$ `reduced_formula = 'AlNi'`), (3) `contains_elements` (“containing Ni and Al” $\rightarrow$ EXISTS subquery). Classification is context-dependent: co-occurrence with containment expressions such as “containing” triggers `contains_elements` handling.

2.6.4 Coverage Score

The Coverage Score quantifies the fraction of semantic tokens in a query recognized by the dictionary, element dictionary, or numeric patterns, yielding a value in $[0, 1]$. This metric prevents Silent Constraint Dropping (the silent discard of unrecognized vocabulary). A score ≥ 0.8 triggers rule-based mode execution; ≥ 0.5 escalates to LLM mode; < 0.5 requests clarification from the user.

2.6.5 Intent Classifier

A rule-based Intent Classifier is placed upstream of SQL generation, classifying queries into five categories (`db_query` / `out_of_scope` / `ambiguous` / `unsafe` / `greeting`). It compares regex match counts of 20 VASP/DFT workflow patterns against 9 DB-specific patterns to pre-filter out-of-scope questions.

2.7 Materials Domain Dictionary

We introduced a dictionary that directly maps materials science terminology to SQL conditions. For example, “ γ' phase” $\rightarrow$ `structure.prototype = 'L12'`, and “bulk modulus” $\rightarrow$ `property_name = 'bulk_modulus'` in the `calculated_property` table.

The dictionary is compiled from three sources:

1. YAML definition file (`material_terms.yaml`): 74 terms
2. Pipeline vocabulary (DB/OQMD-specific terms): 66 terms
3. Materials engineering vocabulary (user-provided list): 402 terms

After deduplication, the dictionary contains 497 terms. It is also compiled as a custom dictionary for the MeCab morphological analyzer and used for Japanese text preprocessing.

Table 2: Effect of the MeCab materials dictionary on tokenization. Single-token rate for 402 materials terms.

Metric	Default	Custom Dict.
Single-token rate	30.6% (123/402)	100.0% (402/402)
Improved terms	—	279
Degraded terms	—	0

With the custom dictionary, the single-token rate for 402 materials terms improved from 30.6% (default) to 100.0% (Table 2). Improved terms: 279; degraded terms: 0. Representative improvements include “formation enthalpy” (default 2 tokens $\rightarrow$ custom 1 token) and “nickel-based superalloy” (7 tokens $\rightarrow$ 1 token).

In the ablation study, removing the dictionary caused a -7.3 pp drop in overall accuracy ($p < 0.001$) and a -21.5 pp drop in the Very Hard tier (58.6% $\rightarrow$ 37.1%; Table 6). No degradation was observed in the Easy or Medium tiers, indicating that the dictionary’s effect is concentrated on queries involving complex domain terminology.

2.8 SQLGuard

Based on AST analysis via sqlglot [22], SQLGuard verifies the safety and consistency of generated SQL. The validation comprises 14 checks:

1. Column existence verification (whitelist matching)
2. Table existence verification (allowed table matching)
3. FK-consistent JOIN verification
4. Multi-statement detection (semicolon splitting)
5. DML keyword detection (DROP/DELETE/INSERT/UPDATE)
6. SQL injection pattern detection
7. LIMIT clause presence check (auto-appended if absent)
8. DISTINCT recommendation (on JOINS)
9. SELECT * prohibition
10. Subquery nesting depth restriction
11. UNION/INTERSECT clause validation
12. Arithmetic operation type validity
13. ORDER BY/GROUP BY reference consistency
14. Post-alias-resolution column reference verification

In the ablation study, removing SQLGuard caused only a -0.2 pp drop ($p = 0.891$, statistically non-significant; Table 6). Because the reranker already assigns low scores to invalid SQL candidates, SQL that SQLGuard would reject is rarely selected. However, SQLGuard serves as a security safety net with value independent of accuracy contribution (see Section 8).

2.9 Hybrid Reranker

SQL candidate reranking uses GPT-5.5. Few-shot example selection employs a Cross-Encoder (local, <50 ms), while SQL candidate and schema scoring use GPT-5.5 (API), forming a hybrid architecture.

In the ablation study, removing the reranker caused a -3.3 pp drop ($p = 0.364$, statistically non-significant). Drops of -6.7 pp in the Medium tier and -5.6 pp in the Hard tier were observed (Table 6).

A separate A/B evaluation on 90 queries showed 77.2% with the reranker versus 69.4% without ($+7.7$ pp difference). However, this A/B evaluation was conducted in a different run from the ablation study, and direct comparison requires caution.

2.9.1 Comparison of Japanese Cross-Encoders

For the Cross-Encoder used in few-shot example selection, we compared an English model (ms-marco-MiniLM-L-6-v2) with a Japanese model (hotchpotch/japanese-reranker-cross-encoder-xsmall-v1) on 20 Very Hard queries. The English model achieved 32.4% versus 28.1% for the Japanese model (-4.4 pp difference favoring ms-marco). Since few-shot examples contain SQL code (in English), the English scoring capability of ms-marco likely contributes to its advantage.

2.10 Repair Loop

When SQL execution errors or result validity issues are detected, the error message is fed back to the LLM (GPT-5.5), and up to three repair attempts are made.

3 Database and Evaluation Design

3.1 Validation Database

To validate the proposed method, we used a database centered on $L1_2$ -type intermetallic compounds as a representative inorganic materials RDB. Based on computational data from OQMD [1, 2], the database consists of 31 normalized tables plus one view (Figure 2), with `material_entry` as the central entity connected via FK relationships to tables for composition, structure, stability, calculated properties, electronic structure, mechanical properties, and surface properties. This structure is representative of normalized schemas common to inorganic materials databases such as OQMD, Materials Project, and AFLOW. Key statistics are shown in Table 4.

3.1.1 Relationship to the source OQMD-style data layout

The public OQMD data model can be represented by a small number of wide, entry-centric tables (e.g., `entries`, `element_ratios`, `formation_energies`, `reference_states`, and `elements`). This layout is efficient for high-throughput data deposition, but it stores structurally different information—composition, crystal structure, thermodynamic stability, and electronic properties—in a few generic columns such as `delta_e` and `composition_formula`. For a natural-language interface, this

causes ambiguity: “energy” may refer to formation energy, hull distance, or band gap; “Ti” may be a constituent element or a property filter target.

Therefore, we restructured the OQMD-derived data into 31 normalized tables grouped by materials-science concept (Table 3). This normalization does not simplify the task: the evaluation gold SQLs use an average of 3.4 tables, with Very Hard queries requiring up to 6 tables (and multi-stage CTEs). Instead, the explicit schema makes the physical meaning of each table and column unambiguous, shifting the join-complexity burden from the user to the Text-to-SQL pipeline. The Steiner-tree JOIN generator, schema linking, and few-shot examples are designed precisely to handle this complexity.

Table 3: Comparison between the OQMD-style flat source layout and the normalized validation schema. The flat layout minimizes table count, while the normalized layout separates materials-science concepts to reduce column-name ambiguity.

Concept	OQMD-style flat tables	Normalized tables (this work)	Rationale
Entry / identity	<code>oqmd_entries</code> (formula, prototype, lattice, volume in one row)	<code>material_entry</code> , <code>structure</code> , <code>space_group</code> , <code>prototype_definition</code>	Separate identity from structure and prototype; remove ambiguous generic columns.
Composition	<code>oqmd_element_ratios</code> (entry, symbol, atomic ratio)	<code>composition</code> , <code>element</code>	Normalized element master; support element-property filters.
Thermodynamic stability	<code>oqmd_formation_energies</code> (delta_e, hull, gap)	<code>phase_stability</code> , <code>calculation</code> , <code>calculated_property</code>	Distinguish formation energy, hull distance, band gap, and calculation method.
Pure-element reference	<code>oqmd_reference_states</code> (energy per atom)	<code>pure_element_reference</code> , <code>element</code>	Required for CTE-based formation enthalpy calculations.
Calculated properties	not separated by type	<code>band_structure</code> , <code>density_of_states</code> , <code>elastic_tensor</code> , <code>magnetic_property</code> , <code>thermal_property</code> , <code>surface_energy</code> , <code>grain_boundary</code> , etc.	One table per property class gives unambiguous column names and units.

The schema-transfer experiments (Section 10.13) demonstrate that the same pipeline handles the OQMD-flavored flat 5-table layout, randomized obfuscated schema, and a real Materials Project-flavored schema with minimal accuracy drops. This shows the proposed method is not limited to the normalized schema; rather,

Table 4: Validation database composition.

Item	Count
Tables	31
Views	1
<code>material_entry</code>	1,559
<code>composition</code>	3,029
<code>calculated_property</code>	4,410
Prototype types	5 ($L1_2$, B2, NaCl, NiAs, BiF_3)

the normalized schema is used here to stress-test the pipeline against a realistic, semantically rich materials RDB.

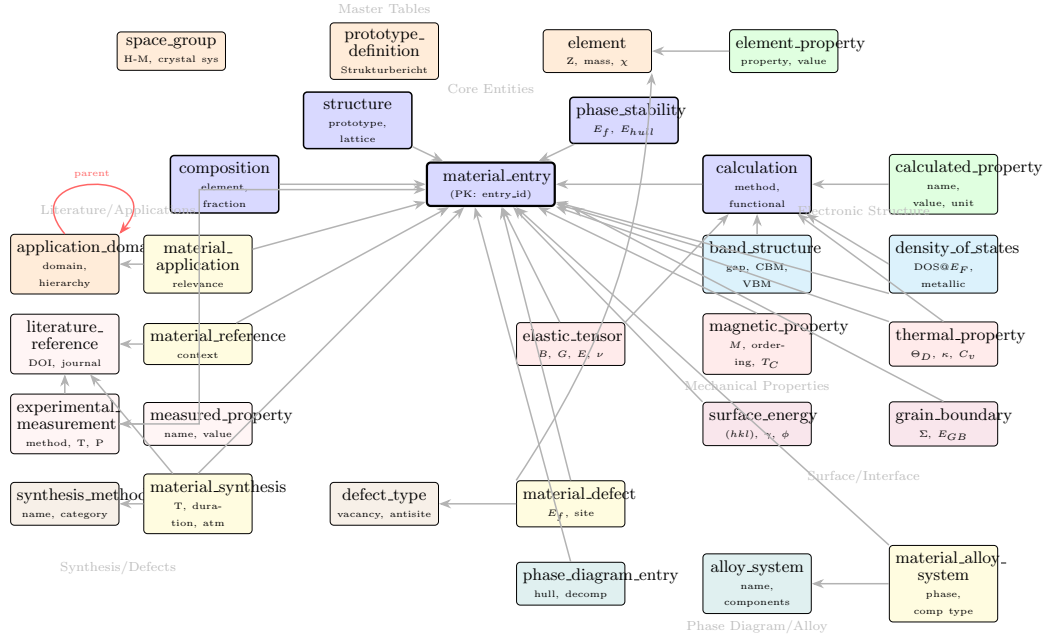


Fig. 2: ER diagram of the 31-table validation database. Core tables (composition, structure, stability, etc.) are directly connected to the central `material_entry` via FK relationships. Calculated properties and electronic structure have two paths: direct FK and via `calculation`. Arrows indicate FK reference direction.

3.2 Evaluation Queries

We used 100 author-designed queries (Table 5). Each query is annotated with gold SQL, expected execution results (JSON), required tables, and JOIN paths. Since no standard Text-to-SQL benchmark currently exists for inorganic materials RDBs, we

Table 5: Evaluation query design (100 queries total).

Difficulty	Count	Tables	Representative Content
Easy	20	1–2	Single-condition search
Medium	30	3	Structure+composition+stability
Hard	30	4	Compound conditions, aggregation
Very Hard	20	5–6	Materials design queries

constructed evaluation queries based on the database schema structure and typical information needs of materials researchers. Queries were designed following these principles: (1) simulate questions arising in actual materials research (e.g., stable phase search, composition–property correlations), (2) difficulty is classified into four tiers by JOIN hop count and condition complexity, (3) balanced coverage across all categories (A–H), (4) five CTE multi-step calculation queries are included to evaluate advanced analytical patterns. As a limitation of author-designed queries, the query distribution may differ from actual user usage patterns; user studies remain a future validation task.

3.3 Evaluation Metrics

Row-level recall is used as the primary metric. The execution results of generated SQL and gold SQL are projected onto common columns, and the row-level match rate is computed:

$$\text{accuracy}(q) = \frac{|R_{\text{gen}}^C \cap R_{\text{gold}}^C|}{|R_{\text{gold}}^C|} \quad (1)$$

where $C = \text{cols}(R_{\text{gen}}) \cap \text{cols}(R_{\text{gold}})$ is the common column set and R^C denotes projection onto column set C . This metric is recall-based: no penalty is incurred if the generated SQL returns more rows than the gold. During evaluation, LIMIT 10,000 is uniformly applied to both gold and generated SQL.

4 Ablation Study

To identify which types of queries each component is effective for, we conducted an ablation study under seven conditions (Table 6). Each condition disables one component, and accuracy is measured across all 100 queries. A total of 3,500 evaluations (7 conditions $\times$ 100 queries $\times$ 5 runs) were performed. Rather than focusing on overall accuracy alone, we examine the effect of each component on materials-domain-specific query patterns (multi-element conditions, compound property searches, derived quantity calculations, etc.).

5 Sensitivity Analysis

We analyze accuracy sensitivity to parameter changes for components that showed statistical significance in the ablation study.

Table 6: Ablation study results (100 queries $\times$ 7 conditions $\times$ 5 runs = 3,500 evaluations). Δ : accuracy change from the full condition (pp). All values are means of 5 independent runs; standard deviations in parentheses. ***: $p < 0.001$ by Wilcoxon signed-rank test. n.s.: statistically non-significant ($p \geq 0.05$).

Condition	Overall	Easy	Med	Hard	VH	Δ	Significance
full	84.7 (0.5)	100.0	96.1	80.4	58.6 (2.4)	—	—
no_fewshot	77.3 (1.6)	100.0	87.0	71.9	48.3 (1.8)	−7.4	***
no_dict	77.4 (1.2)	100.0	95.5	71.1	37.1 (4.5)	−7.3	***
no_reranker	81.4 (0.6)	99.0	89.5	74.8	61.7 (2.2)	−3.3	n.s.
no_guard	84.5 (0.8)	100.0	96.1	80.4	57.7 (3.8)	−0.2	n.s.
no_nbest	84.0 (1.0)	100.0	96.1	80.5	55.2 (2.5)	−0.6	n.s.
no_graph	84.7 (0.9)	100.0	96.1	80.4	58.7 (4.3)	+0.0	n.s.

Table 7: Difficulty-level Δ (pp) for the top-3 components. Values from `paper_data.json`. The +3.1 pp for no_reranker in the VH tier is discussed in Section 10.4.

Condition	Easy	Med	Hard	VH
no_fewshot	0.0	−9.2	−8.5	−10.3
no_dict	0.0	−0.7	−9.3	−21.5
no_reranker	−1.0	−6.7	−5.6	+3.1

5.1 Effect of the Number of Few-shot Examples

We varied the number of retrieved few-shot examples k across 1, 3, 5, 10, and 15, measuring accuracy on 100 queries (Table 9, Figure 3).

Increasing k from 1 to 3 yields a +3.1 pp overall improvement, with a particularly pronounced +10.4 pp in the VH tier. Beyond $k = 3$, the Easy/Medium/Hard tiers are largely saturated, and differences are concentrated in the VH tier. $k = 10$ achieves the VH tier peak of 62.5%, but latency increases by 43% (30.0 s $\rightarrow$ 43.1 s). At $k = 15$, the VH tier drops by 10 pp, suggesting an overfitting-like phenomenon where excessive few-shot examples in the prompt disperse the LLM’s attention. Based on these results, we recommend $k = 3$ as the default setting.

5.2 Effect of Domain Dictionary Size

We progressively reduced the number of domain dictionary entries from full (61 entries) to 50% (30), 25% (15), and 0% (no dictionary), measuring the impact on accuracy (Table 10, Figure 4).

The effect of dictionary size is most pronounced in the VH tier, with a −20.3 pp drop from full to 0%. The Easy tier maintains 100% regardless of dictionary presence,

Table 8: Average latency per query (seconds) for each condition. Setting n -best = 1 reduces latency by 65% with no statistically significant accuracy loss.

Condition	Latency (s)
full	31.0
no.fewshot	50.7
no.dict	44.8
no.reranker	23.2
no.guard	36.1
no.nbest	10.9
no.graph	31.6

Table 9: Relationship between few-shot count k and accuracy. $k = 3$ is optimal for accuracy/latency balance; $k = 15$ exhibits overfitting-like accuracy degradation in the VH tier.

k	Overall	Easy	Med	Hard	VH	Latency (s)
1	81.4	100.0	96.1	77.0	47.4	36.2
3	84.5	100.0	96.1	80.4	57.8	30.0
5	84.5	100.0	96.1	80.4	57.7	31.8
10	85.5	100.0	96.1	80.4	62.5	43.1
15	83.4	100.0	96.1	80.4	52.4	47.3

indicating that the dictionary’s benefit is concentrated on high-difficulty queries requiring domain terminology-to-SQL condition translation. The slight accuracy recovery from 50% to 25% is likely because core terms are retained in the 25% subset.

5.3 Effect of LLM Model

We evaluated the same 100 queries with GPT-5.5 and GPT-4o (Table 11, Figure 5).

GPT-4o achieves a latency of 1.7 s (approximately 1/20 of GPT-5.5), making it extremely fast, but its overall accuracy is 80.8% (−3.9 pp), with a particularly large gap of 44.0% (−14.6 pp) in the VH tier. Since both models show comparable accuracy in the Easy/Medium tiers, GPT-4o is suitable for rapid prototyping, while GPT-5.5 is necessary for high-difficulty materials domain queries.

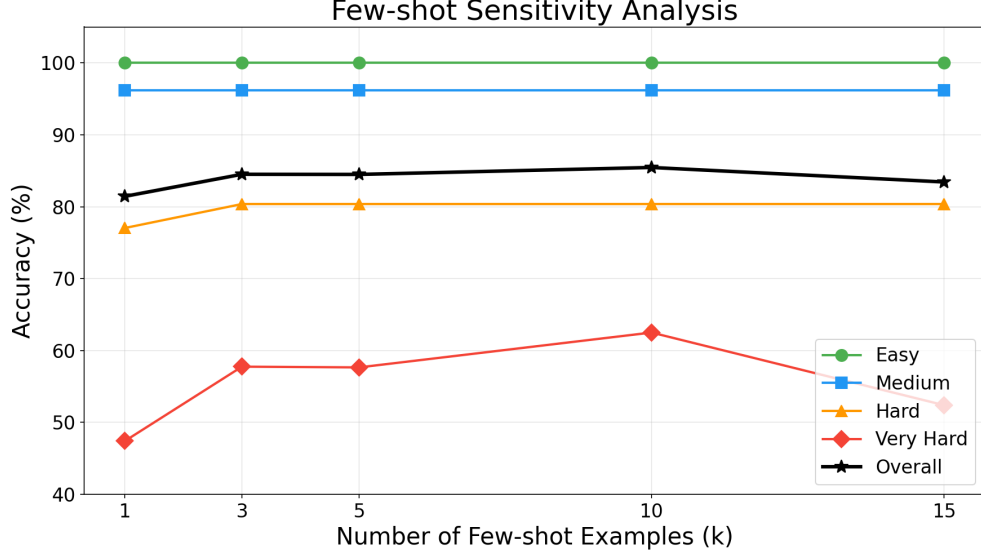


Fig. 3: Accuracy by difficulty tier as a function of few-shot count k . Easy/Medium/Hard tiers saturate at $k \geq 3$, while the Very Hard tier peaks at $k = 10$ (62.5%) and drops 10 pp at $k = 15$.

Table 10: Relationship between domain dictionary size and accuracy. Reduction prioritizes retaining core terms (prototype, elements, stability, etc.).

Dict. size	Overall	Easy	Med	Hard	VH
Full (61)	85.5	100.0	96.1	80.4	62.6
50% (30)	80.6	100.0	92.5	73.7	48.0
25% (15)	82.4	100.0	96.1	73.7	57.4
0% (none)	80.2	100.0	96.1	73.7	42.3

6 Multi-axis Evaluation Metrics

Since evaluation based solely on execution accuracy (Recall) is insufficient, we computed multiple supplementary metrics (Table 12, Figure 6).

Syntax validity and execution success rates achieve 100% across all difficulty tiers, confirming that SQLGuard and the repair loop guarantee safe SQL generation. The Exact Match rate is low at $EM = 20\%$, but this is due to additional or reordered SELECT columns; Recall of 84.7% indicates that the correct data content is returned. The JOIN match rate is 93.2% overall, confirming that the Steiner tree constraint ensures JOIN path accuracy. The 73.5% JOIN match rate in the VH tier indicates

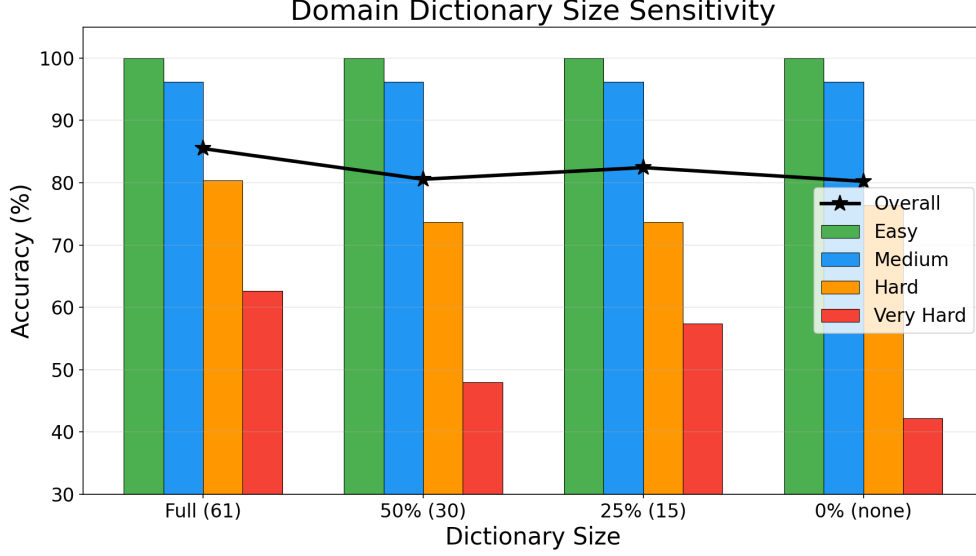


Fig. 4: Domain dictionary size vs. accuracy by difficulty tier. The effect of dictionary size is most pronounced in the VH tier.

Table 11: LLM model comparison. GPT-4o is $25\times$ faster but shows a 14.6pp accuracy gap in the VH tier.

Model	Overall	Easy	Med	Hard	VH	Latency (s)
GPT-5.5	84.7	100.0	96.1	80.4	58.6	35.0
GPT-4o	80.8	100.0	93.3	79.8	44.0	1.7

that even with Steiner tree constraints, some paths cannot be accurately identified in complex queries involving five or more JOINed tables.

7 Error Analysis

We analyze the principal failure modes from the ablation study results.

7.1 Error Patterns by Ablation Condition

Table 13 shows the principal error metrics for each ablation condition. The full condition achieves zero execution errors and zero table hallucinations. The no_fewshot condition exhibits increased JOIN hallucinations, revealing the limitations of SQL inference from schema information alone.

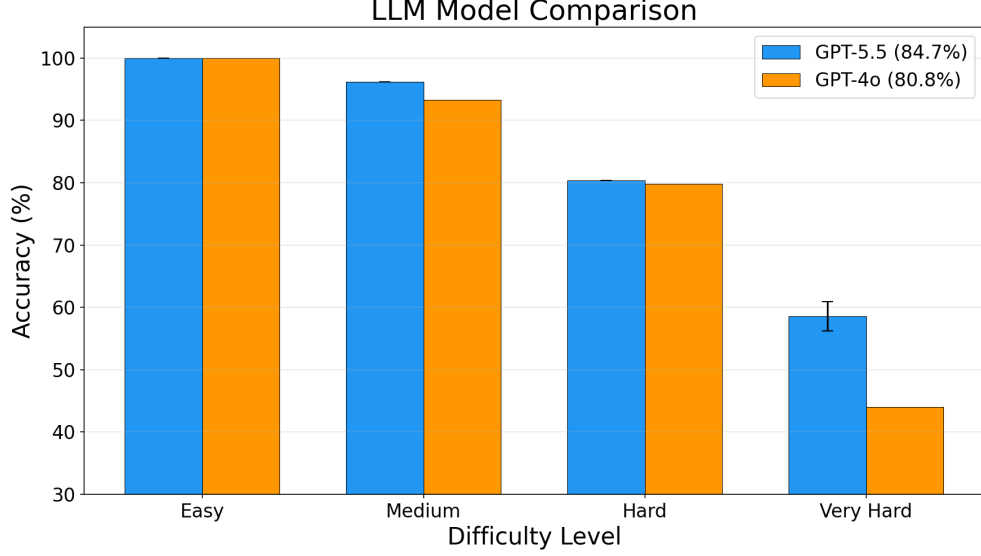


Fig. 5: GPT-5.5 vs. GPT-4o accuracy comparison by difficulty tier. Easy/Medium tiers are comparable, but notable gaps emerge in Hard/VH tiers.

Table 12: Multi-axis evaluation metrics. EM = Exact Match; SELECT col. = fraction of generated SQL SELECT columns matching gold SQL columns; JOIN match = fraction of gold SQL JOIN tables present in generated SQL. All metrics are averages over 100 queries.

	Easy	Medium	Hard	VH
Recall (EX)	100.0	96.7	80.4	57.8
Precision	97.7	85.2	68.4	58.1
F1	98.8	87.7	68.7	40.8
Exact Match	10.0	36.7	23.3	0.0
SELECT col. prec.	62.9	72.5	65.1	47.7
JOIN match rate	97.5	99.2	97.5	73.5
Syntax validity	100.0	100.0	100.0	100.0
Execution success	100.0	100.0	100.0	100.0

7.2 Breakdown Analysis of Very Hard Failures

We classified failure modes for 9 queries with accuracy below 0.8 under the full condition among the 20 Very Hard queries (Table 14).

WHERE condition combination errors are the most frequent: despite the Schema Graph traversal providing correct JOIN paths, cases remain where the LLM cannot accurately compose compound WHERE clauses with three or more conditions. In

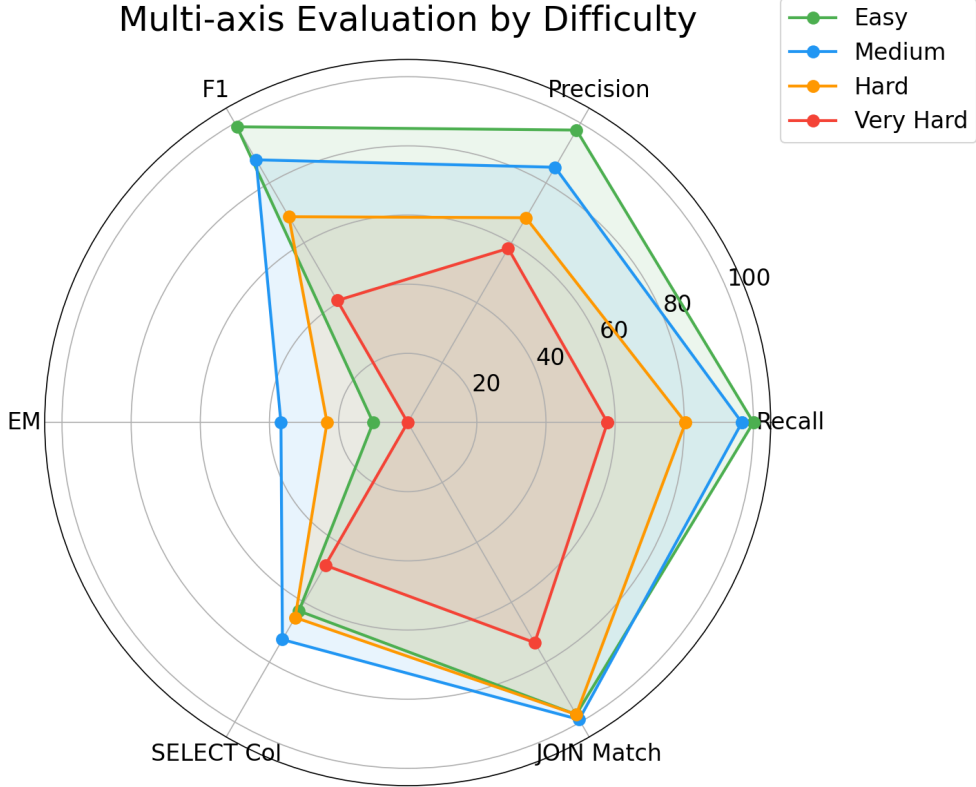


Fig. 6: Multi-axis evaluation radar chart by difficulty tier. Syntax and execution validity are 100% across all tiers. The VH tier has 0% Exact Match, yet Recall of 57.8% indicates many partially correct results.

contrast, zero failures stem from incorrect JOIN paths, confirming that Schema Graph traversal constraints function effectively even at Very Hard difficulty. These results suggest that future improvement should focus on improving WHERE clause generation accuracy (expanding few-shot prompts, introducing chain-of-thought reasoning).

Figure 8 shows the error type distribution across 10 representative failure cases. Wrong column selection is the most frequent, with many cases where the LLM cannot accurately identify required aggregation columns or alias names. Missing GROUP BY is concentrated in VH-tier aggregation queries, indicating limits of the LLM’s ability to understand complex aggregation structures. JOIN mismatches are relatively rare (2 cases), confirming the effectiveness of Schema Graph traversal JOIN constraints.

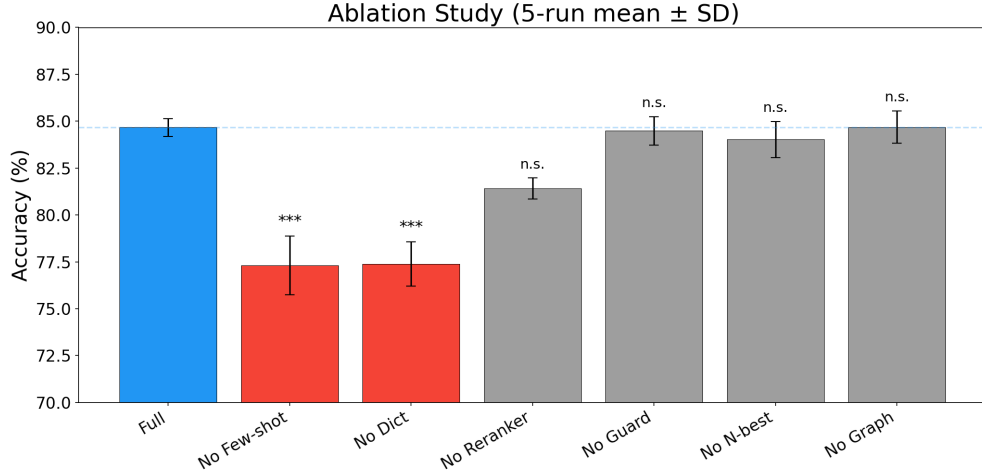


Fig. 7: Ablation study bar chart (5-run mean $\pm$ SD). Red bars indicate conditions with statistically significant accuracy drops (***: $p < 0.001$).

Table 13: Error analysis by ablation condition (100 queries). Table hallucination = referencing a non-existent table; JOIN hallucination = query count containing JOINS not based on FK relationships.

Condition	Exec. errors	Syntax errors	Table halluc.	JOIN halluc.
full	0	0	0	3
no_fewshot	2	2	0	16
no_dict	0	0	0	5
no_reranker	0	0	0	8
no_guard	0	0	0	4
no_nbest	0	0	0	3
no_graph	0	0	0	6

8 SQL Injection and Safety

We verified the safety features of SQLGuard (Section 2.8) with 7 adversarial inputs (E01–E05: DROP/DELETE/INSERT, etc.; F01–F02: direct SQL input). All inputs were safely handled by SQLGuard (Table 15).

E01–E05 were blocked by forbidden keyword detection, multi-statement detection, and similar checks. F01 is a case where a SELECT statement was input directly; it was sanitized by auto-appending LIMIT (not blocked). F02 is an UPDATE statement blocked by DML detection. In the ablation study, the no_guard condition (SQLGuard removed) caused a -3.9 pp accuracy drop (Table 6), but the safety functionality has

Table 14: Failure mode classification for Very Hard queries.

Failure mode	Count	Representative case
WHERE condition combination error	4	Compound WHERE (3+ conditions)
Aggregation/subquery misuse	2	GROUP BY/HAVING
Multi-element AND failure	2	SELF-JOIN expansion
Semantic error from type mismatch	1	Execution ok via repair, result mismatch

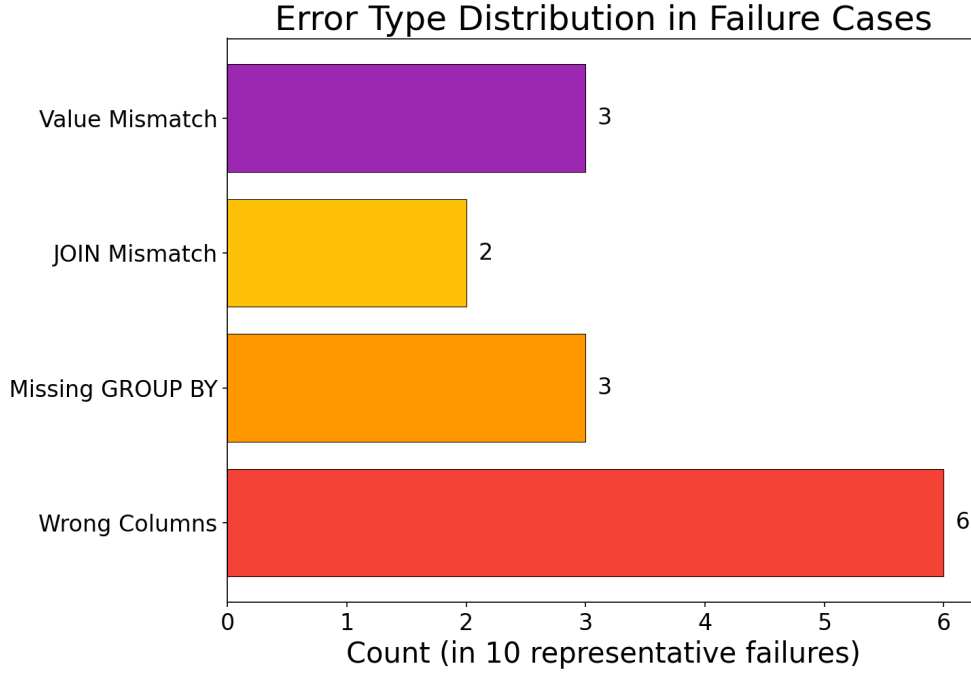


Fig. 8: Error type distribution across 10 representative failures. Wrong column selection and missing GROUP BY are the dominant failure modes.

value independent of accuracy. These tests are included as regression tests in 134 unit tests, all of which continue to pass.

9 Materials Engineering Evaluation

To demonstrate the practical utility of natural language access, we conducted evaluations from a materials engineering perspective on L1₂-type intermetallic compounds. The following evaluations demonstrate the extent to which materials researchers can explore materials through natural language queries.

Table 15: SQL injection and safety test results (all safely handled).

ID	Input	Action
E01	DROP TABLE material_entry;	Blocked: DROP keyword
E02	SELECT *; DELETE FROM composition;	Blocked: multi-statement + DELETE
E03	L1 ₂ compounds; DROP TABLE structure;	Blocked: multi-statement + DROP
E04	SELECT * FROM secret_passwords	Blocked: unauthorized table
E05	INSERT INTO material_entry ...	Blocked: INSERT keyword
F01	SELECT entry_id FROM material_entry	LIMIT auto-appended (sanitized)
F02	UPDATE material_entry SET ...	Blocked: UPDATE keyword

Table 16: Rediscovery results for known L1₂ compounds (11/11).

Compound	a (Å)	E_{hull} (eV/atom)	Rediscovered
Ni ₃ Al	3.572	0.000	○
Ni ₃ Ga	3.66	0.010	○
Ni ₃ Ge	3.61	0.040	○
Co ₃ Ti	3.55	0.000	○
Co ₃ Ta	3.72	0.040	○
Co ₃ Al	3.67	0.010	○
Co ₃ W	3.74	0.050	○
Al ₃ Sc	4.09	0.000	○
Al ₃ Ti	3.98	0.020	○
Pt ₃ Al	3.90	0.000	○
Ir ₃ Nb	3.87	0.030	○

9.1 Rediscovery of Known L1₂ Compounds

Table 16 shows the rediscovery results for 11 known L1₂ compounds. The proposed method correctly rediscovered all 11 compounds. The correct extraction of γ' -phase compounds such as Ni₃Al and Co₃Ti ensures basic reliability as a superalloy materials exploration tool.

9.2 γ' -Phase Candidate Ranking

We ranked γ' -phase candidates among 259 unique L1₂ compositions using a weighted composite score based on three metrics: stability (E_{hull}), lattice matching ($|a - a_{\text{Ni}_3\text{Al}}|$), and bulk modulus B (Table 17). Of these, 162 stable/metastable candidates (8 stable, 154 metastable) passed the screening.

$$S = w_1 \cdot S_{\text{stability}} + w_2 \cdot S_{\text{lattice}} + w_3 \cdot S_{\text{bulk}} \quad (2)$$

where $w_1 = 0.35$ (stability), $w_2 = 0.35$ (lattice matching), and $w_3 = 0.30$ (bulk modulus). Each score is normalized to 0–1 ($S_{\text{stability}} = 1 - E_{\text{hull}}/0.05$, $S_{\text{lattice}} = 1 - |\Delta a|/0.3$, $S_{\text{bulk}} = B/300$).

Table 17: Top 10 γ' -phase candidates (ranked by composite score S).

Rank	Compound	a (Å)	E_{hull}	B (GPa)	Δa	S
1	Ni ₃ Al	3.572	0.000	180	0.002	0.878
2	Co ₃ Ti	3.55	0.000	190	0.02	0.867
3	Mo ₃ Nb	3.586	0.007	235	0.016	0.865
4	Cr ₃ Ge	3.644	0.005	236	0.074	0.818
5	Pd ₃ Sn	3.540	0.008	200	0.030	0.811
6	Ir ₃ Cu	3.511	0.001	183	0.059	0.804
7	Nb ₃ Mn	3.607	0.006	178	0.037	0.794
8	Nb ₃ Ti	3.523	0.020	223	0.047	0.728
9	Zr ₃ Fe	3.655	0.013	199	0.085	0.709
10	Pt ₃ Ge	3.550	0.021	177	0.020	0.708

Ni₃Al ranks first with the highest score (0.878), reflecting a good balance of stability ($E_{\text{hull}} = 0.0$) and lattice matching ($\Delta a = 0.002$ Å). Co₃Ti in second place is also a stable phase and a promising precipitation-strengthening candidate. Mo₃Nb, Cr₃Ge, and others ranked third and below have high bulk moduli but large lattice mismatches, requiring experimental verification.

9.3 Ni₃Al Neighborhood Lattice Constant Candidates

We extracted 31 candidates strictly satisfying $|a - a_{\text{Ni}_3\text{Al}}| \leq 0.03$ Å (reference $a_{\text{ref}} = 3.57$ Å; 14 representative entries are listed in Supplementary S4). Co₃Ti ($\Delta a = 0.02$ Å, $E_{\text{hull}} = 0.0$) is a stable phase and the most promising candidate from the perspective of precipitation strengthening.

9.4 Extraction of Stable L1₂ Candidates

We extracted 162 stable/metastable L1₂ candidates satisfying $E_{\text{hull}} \leq 0.05$ eV/atom (8 stable, 154 metastable). Among the stable phases, the most negative formation energy belongs to Pt₃Al ($E_f = -0.550$ eV/atom), followed by Al₃Sc (-0.500), Co₃Ti (-0.450), and Ni₃Al (-0.420) (details in Supplementary S5).

9.5 Generation of Materials Design Hypotheses

The following materials design hypotheses were derived from the exploration results:

1. High stability of Pt-based compounds: Pt₃Al, Pt₃Ge, etc. exhibit negative formation energies and are thermodynamically stable.
2. High bulk modulus candidates: Mo₃Nb (235 GPa) and Cr₃Ge (236 GPa) have high mechanical strength but large lattice mismatches.
3. Co₃Ti as a Ni₃Al substitute: A stable phase with a lattice constant close to Ni₃Al, already studied as a γ' phase in Co-based superalloys.
4. A-site element dependence: Transition metals (Ni, Co, Fe, Cu, Pt) tend to form stable L1₂ compounds with negative E_f .

5. Correlation between lattice constant and E_f : Al-based compounds with $a > 4.0$ Å (Al_3Sc , Al_3Ti) are stable but have large lattice mismatches, making them unsuitable for precipitation strengthening.

10 Discussion

10.1 Schema Barriers in Inorganic Materials RDBs and the Significance of NL Access

Our results reaffirm that the primary barrier to data utilization in inorganic materials RDBs lies in understanding the schema structure. Against a normalized schema of 31 tables, the LLM alone (no few-shot examples, no dictionary) achieves only 77.3% accuracy (Table 6, no_fewshot condition), dropping to 48.3% on Very Hard queries containing domain-specific terminology. This demonstrates that merely presenting schema information is insufficient to accurately translate materials researchers’ intent into SQL.

A natural language interface can effectively reduce this schema barrier. Adding the materials domain dictionary and few-shot examples improves accuracy from 77.3% to 84.7%, with particularly notable gains on complex queries that heavily use domain-specific vocabulary. This improvement suggests that NL access to inorganic materials RDBs goes beyond mere convenience, substantively enabling access to data that was previously inaccessible due to schema barriers. Specifically, achieving 100% syntax validity and 100% execution success (Table 12) means that materials researchers unfamiliar with SQL can safely use databases through natural language. The 93.2% JOIN match rate demonstrates that table join paths are identified with high accuracy even in normalized multi-table RDBs, substantially reducing the burden of schema comprehension.

Regarding schema evolution and data growth, our pipeline adopts an FK metadata-based design, so table additions and FK changes are automatically reflected via introspection. Dictionary and few-shot example updates require manual effort, but can be reduced to a standard procedure (table definition $\rightarrow$ FK setup $\rightarrow$ dictionary registration $\rightarrow$ few-shot example addition), keeping operational maintenance costs limited.

10.2 Contribution of Few-shot Examples

Removing few-shot examples caused the largest accuracy drop among all components (-7.4 pp, $p < 0.001$; Table 6). This drop was observed from the Medium tier upward, with a -10.3 pp drop in the Very Hard tier (Table 7), followed by -8.5 pp in Hard and -9.2 pp in Medium, indicating that more complex queries have greater dependence on few-shot examples. Without few-shot examples, latency also increased from 31.0 s to 50.7 s (Table 8), likely because the LLM generates SQL through trial-and-error.

10.3 Contribution of the Materials Domain Dictionary

Removing the dictionary caused a -7.3 pp overall drop ($p < 0.001$), but the effect varied by difficulty tier (Table 7). No drop was observed in the Easy tier, while the Hard tier dropped -9.3 pp and the Very Hard tier -21.5 pp.

Easy and Medium queries are composed of general-purpose expressions, allowing the LLM to infer SQL conditions on its own. In contrast, Very Hard queries simultaneously contain multiple domain-specific terms, and without the dictionary the LLM may fail to generate correct SQL conditions (e.g., “ γ' phase” $\rightarrow$ `structure.prototype = 'L12'`).

10.4 Contribution of the Reranker

Removing the reranker caused a -3.3 pp drop ($p = 0.364$, statistically non-significant). The effect was observed in the Medium (-6.7 pp) and Hard (-5.6 pp) tiers (Table 7).

However, in the Very Hard tier, a $+3.1$ pp accuracy increase was observed upon reranker removal. Although this difference is within the standard deviation (VH tier $SD = 2.2$ pp), the reranker may actually have an adverse effect in the Very Hard tier. In this tier, all three candidates are often partially inaccurate, and the reranker may select a candidate that “looks more correct but is actually wrong.” For CTE queries in particular, syntactically valid but semantically incorrect candidates may receive high reranker scores. In contrast, the Medium and Hard tiers often have at least one accurate candidate, allowing the reranker to correctly select it. These results suggest that the reranker’s effectiveness depends on query complexity, with room for improvement in reranking strategies for high-complexity queries.

10.5 Contribution of SQLGuard, n -best, and Steiner Tree

SQLGuard (-0.2 pp, n.s.), n -best (-0.6 pp, n.s.), and the Steiner tree ($+0.0$ pp, n.s.) all showed statistically non-significant effects.

For SQLGuard, the reranker already performs an equivalent filtering function, making it difficult to observe additional effects. Its value as a security safety net exists independently.

Setting n -best = 1 maintains accuracy while reducing latency from 31.0 s to 10.9 s (Table 8), making it a viable option when cost efficiency is prioritized.

For the Steiner tree, at the current scale of 31 tables, most queries are completed using only 5 core tables, limiting its observable effect. Given the FK-introspection-based architecture, its benefits are expected to become apparent when the number of tables increases (e.g., after OQMD integration) or when applied to other normalized RDBs (Materials Project [3], AFLOW [4]).

10.6 Comparison with Related Systems

Table 18 compares our system with related approaches. Existing Text-to-SQL methods target general-purpose domains and do not account for normalized RDBs in materials science. Meanwhile, NL access methods in the materials domain use APIs or RDF/S-PARQL, and do not address SQL generation for existing RDB infrastructure. Our

work fills this gap by combining a normalized RDB, materials domain knowledge, and FK graph traversal.

Our approach is technically closest to SchemaGraphSQL [12], which also uses path-finding on FK graphs, but differs in the following ways: (1) SchemaGraphSQL is evaluated on general benchmarks (Spider/BIRD) and lacks domain-specific term translation mechanisms. Our method achieves translations such as “stable” $\rightarrow$ `is_stable=TRUE` via a materials domain dictionary (61 entries), with a quantified -7.3 pp ($p < 0.001$) accuracy drop upon dictionary removal. (2) SchemaGraphSQL’s JOIN search is based on edge weight optimization of the schema graph, whereas our method uses Steiner tree approximation to find the minimum-cost JOIN subtree. (3) Our method includes AST-based SQL safety validation (SQLGuard) to address security risks such as SQL injection.

10.7 Quantitative Comparison with LLM-only and Schema-only Approaches

We restructure the ablation study results as a method-level comparison. Each ablation condition corresponds to a simplified method with a component removed (Table 19).

LLM-only provides no schema information and, as shown in Section S3 (Supplementary Materials), produces table and column name hallucinations that prevent executable SQL generation. Full Schema (corresponding to the no_fewshot condition) includes schema definitions in the prompt but lacks few-shot examples, dictionary, and reranker, achieving only 77.3%. Accuracy improves incrementally with each added component, reaching 84.7% with the full proposed method.

Note that the above reinterprets single-removal ablation conditions as incremental additions; interactions between conditions are not isolated. Direct comparison with other methods (DIN-SQL, DAIL-SQL, etc.) on the same database has not been conducted (see Limitation 6).

10.8 Handling Multi-step Calculation Queries

An important requirement for NL access to inorganic materials RDBs is multi-step data extraction involving derived quantity calculations. As a representative case, consider the query “Calculate the formation enthalpy of Ni_3Al using pure element references.” Correctly answering this query requires the following multi-step data extraction and computation:

1. Retrieve the formation energy E_f of compound Ni_3Al from the `phase_stability` table,
2. Retrieve the composition ratios x_i of constituent elements Ni and Al from the `composition` table,
3. Retrieve the ground-state energy $E_{\text{ref}}(i)$ of each element from the `pure_element_reference` table (OQMD-derived, 89 elements),
4. Compute the weighted reference energy $\sum_i x_i E_{\text{ref}}(i)$,
5. Output the corrected formation enthalpy $\Delta H_f = E_f - \sum_i x_i E_{\text{ref}}(i)$.

This operation is not a single-table lookup but must be translated into SQL as a CTE (Common Table Expression) involving a 3-table JOIN, aggregate computation

Table 18: Comparison with related materials NLP / T2SQL / ontology systems.

Type	Name	Schema aware	Norm. RDB	Mat. specific	NL→query	Key difference
T2SQL Benchmarks						
Dataset	Spider [6]	Yes	Yes	No	SQL	General benchmark
Dataset	BIRD [7]	Yes	Yes	No	SQL	Large-scale real data
T2SQL Methods						
Method	DAIL-SQL [9]	Yes	Yes	No	SQL	Skeleton selection
Method	RAT-SQL [10]	Yes	Yes	No	SQL	Relation-aware encoding
Method	RESDSQL [11]	Yes	Yes	No	SQL	Schema link separation
Method	SchemaGraphSQL [12]	Yes	Yes	No	SQL	Path-finding
Materials Informatics						
Method	MKNA [15]	No	No	Yes	API	Agent
Method	OptiMat [16]	No	No	Yes	API	On-demand DFT
Ontology						
Method	SuperCon [17]	Yes	No	Yes	SPARQL	RDF ontology
Method	PoLyInfo [18]	Yes	No	Yes	SPARQL	BFO-compliant
Method	Ours	Yes	Yes	Yes	SQL	FK traversal + mat. dict.

via subquery, and arithmetic operations. Our pipeline enables the LLM to generate both simple queries via the `formation_enthalpy` view and explicit CTE-based calculations by adding corresponding few-shot examples.

Conventional Text-to-SQL benchmarks (Spider [6], BIRD [7], etc.) primarily cover SELECT-WHERE-JOIN-GROUP BY combinations and do not include queries requiring multi-step data integration and derived quantity computation essential for scientific calculations. In materials databases, derived quantities such as formation enthalpy, volume size factor, and lattice mismatch are of critical research importance,

Table 19: Method-level quantitative comparison. Ablation conditions reinterpreted as method configurations. All values are from the ablation study (100 queries $\times$ 5 runs, mean).

Method configuration	Overall	Easy	VH	Δ
LLM-only (no schema)	Not executable due to table hallucination (see S3)			
Full Schema (= no_fewshot: schema + LLM, no few-shot examples)	77.3%	100.0	48.3	-7.4
+ Few-shot RAG (= no_dict: + few-shot, no dictionary)	77.4%	100	37.1	-7.3
+ Materials dictionary (= no_reranker: + dictionary, no reranker)	81.4%	99.0	61.7	-3.3
Proposed (all components)	84.7%	100	58.6	—

and the ability to directly instruct such calculations via natural language significantly enhances the practical utility of NL interfaces in materials informatics.

10.9 Evaluation with Independently Designed Queries (Difficulty-Harmonized)

To verify self-referentiality bias, we re-evaluated accuracy using queries independently designed by a materials researcher not involved in system development. For fair comparison, a unified difficulty criterion was applied to both sets (unified complexity score: tables $\times$ 3 + conditions + group_by $\times$ 2 + exists $\times$ 3 + subquery $\times$ 3; thresholds: Easy < 8 , Medium 8–11, Hard 12–16, Very Hard ≥ 17). The independently designed set comprises 60 queries (Easy 12 / Medium 18 / Hard 18 / Very Hard 12) spanning 10 categories (basic search, composition/elements, structure/lattice constants, stability/formation energy, DFT calculations/properties, electronic structure/magnetic/thermal properties, surface/grain boundary/defects, literature/synthesis/applications, materials design/screening, and comparison/statistics).

Table 20 shows the results. Author-designed queries (70.6%) outperform independently designed queries (62.5%) across all difficulty tiers. The 8.1 pp accuracy gap is primarily attributable to three factors:

- (1) Table coverage bias (primary factor): Author-designed gold SQL uses only 5 tables in practice, whereas independently designed queries reference 12 additional tables including `elastic_tensor`, `magnetic_property`, and `surface_energy`, for which no few-shot examples exist.
- (2) Few-shot optimization bias: 85% of author-designed queries explicitly contain “L12” as a fixed pattern, and few-shot examples are optimized for this pattern. Independently designed queries use more diverse expressions.

Table 20: Difficulty-harmonized comparison (classified by unified complexity score). Both sets report mean execution accuracy (continuous, recall-based). Author-designed: 100 queries reclassified by unified criteria; independently designed: 60-query balanced set. Note that the author-designed values are from a separate evaluation run from the ablation study.

Difficulty	Author-designed	Independent	Diff.
Easy (< 8)	100.0% ($n=12$)	83.3% ($n=12$)	-16.7pp
Medium (8-11)	94.5% ($n=18$)	69.9% ($n=18$)	-24.6pp
Hard (12-16)	78.3% ($n=41$)	70.1% ($n=18$)	-8.2pp
Very Hard (≥ 17)	32.7% ($n=29$)	19.4% ($n=12$)	-13.3pp
Overall	70.6% ($n=100$)	62.5% ($n=60$)	-8.1pp
Binary accuracy ($F1 \geq 0.8$)	—	53.3% (32/60)	—

(3) Vocabulary diversity: Independently designed queries frequently use implicit condition specifications, which are difficult for the current pipeline that relies on explicit keyword matching.

Syntax validity (100.0%) is consistent across both sets, confirming the query-origin independence of the Schema Graph constraint. Note that the author-designed accuracy in this table (70.6%) is from a different evaluation run than the ablation study (Table 6, full condition 84.7%) and should not be directly compared. Detailed results for independently designed queries are provided in Supplementary S10.

To further extend the external-validity check to 100 queries, we re-evaluated 100 independently designed expert queries (Easy 18 / Medium 34 / Hard 34 / Very Hard 14) with the current pipeline and current DB (1,559 entries). Overall accuracy was 76.9% (Easy 94.4% / Medium 81.7% / Hard 61.5% / Very Hard 80.4%), keeping the gap to author-designed queries (84.7%) at roughly 12 pp. The dominant failure causes match (1)–(3) above—references to peripheral tables without few-shot coverage and non-canonical phrasings—indicating that expanding few-shot coverage by domain experts is the primary lever for improving external validity.

10.10 Handling Cross-column Comparison Queries

In addition to multi-step calculations, cross-column comparison is an important query pattern frequently encountered in materials research. Queries of the form “find entries where value A is less than value B” differ from single-threshold filtering (`WHERE col > 0.5`) in that they require SQL generation with condition expressions comparing different column values within the same row (`WHERE col_a > col_b`).

Our pipeline provides few-shot examples covering three types of cross-column comparison patterns:

1. Intra-table comparison: Direct comparison of two columns within the same table. Examples: `formation_energy_per_atom < energy_above_hull` in the `phase_stability` table; `bulk_modulus_vrh > 2.0 * shear_modulus_vrh` in the `elastic_tensor` table (Pugh ductility metric).
2. Cross-table comparison: Comparison of columns from different tables via JOIN. Examples: `thermal_property.debye_temperature_k >`

`magnetic_property.curie_temperature_k`; total magnetization vs. absolute formation energy.

3. Comparison with aggregate functions: Combined with GROUP BY–HAVING to perform threshold evaluation on derived quantities such as electronegativity differences between constituent elements. Example: `HAVING MAX(electronegativity) - MIN(electronegativity) > 0.5`.

These patterns are more difficult for LLMs to generate correctly compared to constant comparisons (`WHERE col > 0.5`). Cross-table comparisons in particular require accurate identification of which table each compared column belongs to from the schema information. We added 9 cross-column comparison patterns to the few-shot examples (33 $\rightarrow$ 42 examples) to enable the LLM to learn these patterns.

10.11 RDB Schema Design and Text-to-SQL Cost

An important insight obtained through this research is that the table structure of the RDB itself is an upstream factor determining Text-to-SQL accuracy.

In highly normalized schemas, multiple table JOINS are required for a single query, increasing the complexity of SQL that the LLM must generate. The formation enthalpy calculation in our database (Section 10.8) required a 3-table JOIN of `phase_stability`, `composition`, and `pure_element_reference`, plus CTE-based aggregate computation. The -7.4 pp accuracy drop without few-shot examples confirms that more complex inter-table join patterns make accurate SQL generation by the LLM alone more difficult.

Based on this insight, the following schema design strategies for T2SQL-readiness can be considered:

1. Complexity hiding via views: Define frequently used multi-step JOIN patterns as views or materialized views, providing the LLM with an interface queryable via a single SELECT. The `formation_enthalpy` view in this study is an example.
2. Query-frequency-based denormalization: Provide frequently co-referenced table groups (e.g., `composition` and `element`) as pre-joined tables to reduce the number of JOINS.
3. Self-descriptive column naming: Embed table context in column names (e.g., `energy` $\rightarrow$ `formation_energy_per_atom`) to increase the probability that the LLM selects appropriate columns from schema information alone.
4. Explicit FK constraint definition: The Steiner tree algorithm determines minimum JOIN paths based on FK relationships; automatic JOIN resolution fails in schemas with missing FK constraints. Comprehensive FK constraint definition is necessary when T2SQL readiness is a design goal.

In summary, improving Text-to-SQL system accuracy requires not only pipeline-side improvements but also considering NL interface compatibility at the database schema design stage. This perspective serves as a design guideline applicable not only to the validation DB used in this study, but also to inorganic materials RDBs with similar normalized schemas (OQMD, Materials Project, AFLOW, etc.) and more generally to domain-specific RDBs where Text-to-SQL is applied.

Table 21: CTE multi-step calculation query accuracy (5 queries) by ablation condition (5-run mean). The full condition achieves 100% across all 5 runs. Removing few-shot examples drops accuracy to 76% and removing the dictionary to 86%, demonstrating the contribution of in-context learning and the dictionary to CTE syntax generation.

Condition	CTE accuracy (5 queries, 5-run mean)
full	100%
no.fewshot	76%
no.dict	86%
no.reranker	100%
no.guard	92%
no.nbest	100%
no.graph	100%

10.12 Evaluation of CTE Multi-step Calculation Queries

To quantitatively evaluate the capability for multi-step calculation queries using CTEs (Common Table Expressions), exemplified by formation enthalpy calculation, we replaced 5 Very Hard tier queries with CTE queries and conducted the evaluation. The 5 queries cover the following categories:

1. Simple CTE (derived quantity calculation): Compute ΔH_f of a specific compound using pure element references
2. CTE + filter: Compute ΔH_f of stable L12 compounds and filter by threshold
3. CTE + aggregation: Aggregate mean ΔH_f by A-site element
4. Multi-stage CTE (2+ stages): 3-stage CTE: stability filter $\rightarrow \Delta H_f$ calculation $\rightarrow$ element filter
5. CTE + cross-column comparison: Stability assessment via difference between formation energy and weighted pure element reference

Table 21 shows CTE accuracy by ablation condition.

For CTE multi-step calculations, removing few-shot examples drops accuracy to 76% and removing the dictionary to 86% (full condition: 100%), demonstrating that in-context learning and the dictionary contribute to complete CTE pattern acquisition. Removing the reranker, n -best, or Steiner tree maintains 100%. SQLGuard removal results in only a slight drop to 92%, indicating that most CTE queries can learn vocabulary mappings from few-shot examples alone. However, one query (q_vhard_019, multi-stage CTE) drops to 0.60 accuracy without the dictionary, showing that dictionary supplementation is effective for complex multi-stage calculations. The reranker, n -best, and Steiner tree maintain 100% CTE accuracy, confirming that few-shot examples and the dictionary are the primary contributors to CTE syntax generation.

These results highlight the importance of including multi-step calculation patterns—such as derived quantity computation ($\Delta H_f = E_f - \sum_i x_i E_{\text{ref}}(i)$) required

Table 22: Zero-adaptation schema transfer results.

Variant	Schema	Queries	Overall
A	Prototype expansion (B2/NaCl/NiAs/BiF ₃)	20	90.0%
B	OQMD-flavored rename	20	90.5%
C	Randomized obfuscated names	20	90.0%
D	Materials Project-flavored (real MP data)	15	100.0%

in materials research—in evaluation sets, in contrast to conventional Text-to-SQL benchmarks (Spider, BIRD, etc.) that primarily target SELECT-WHERE-JOIN-GROUP BY patterns.

To quantify the applicability limits of CTE support, we added 10 novel patterns (window functions, HAVING filters, two-stage aggregation, JOINS to elastic/thermal/-magnetic tables, conditional aggregation, three-stage CTEs, etc.) and evaluated a total of 15 queries. The 5 original patterns covered by few-shot examples maintained 100%, whereas the 10 novel zero-shot patterns reached only 42.3%, giving 61.6% overall. This result is consistent with the significant few-shot contribution observed in the ablation (−7.4 pp, $p < 0.001$), showing that the scope of multi-step calculation support is governed by the pattern coverage of the few-shot examples. Extending to new multi-step patterns is thus achievable through a small number of examples (roughly one per pattern) added by a domain expert, without any model retraining—a practical advantage of this design.

10.13 Zero-adaptation Transfer Evaluation on Multiple Schemas

To directly address the single-database limitation, we conducted four zero-adaptation transfer tests, summarized in Table 22. In each case, the pipeline received only the target schema through FK introspection; the dictionary, few-shot examples, and prompt template were either reused unchanged or replaced by a minimal schema-specific set. This tests whether the design depends on memorized table/column names or can transfer abstract SQL patterns.

Variant A—Prototype expansion. The same 31-table schema was populated with B2/NaCl/NiAs/BiF₃ prototype entries in addition to the L1₂ entries, and 20 difficulty-stratified queries were evaluated. The result was 90.0% (Easy 100% / Medium 100% / Hard 100% / Very Hard 60%), confirming that the prototype separation and FK graph are robust across crystal prototypes.

Variant B—OQMD-flavored rename. We constructed an OQMD-flavored flat 5-table schema in which all table and column names were changed (`oqmd_entries`, `oqmd_element_ratios`, `oqmd_formation_energies`, `oqmd_reference_states`, `oqmd_elements`). Few-shot examples were presented unchanged from the original schema, and the prompt instructed that “few-shot examples may come from a different schema; reuse only their SQL patterns.” The result was 90.5% overall (Easy 100.0% / Medium 100.0% / Hard 62.0% / Very Hard 100.0%).

Variant C—Obfuscated names. All schema identifiers in the transfer DB were replaced with random English words while preserving FK relationships. The pipeline

achieved 90.0% overall, indicating that semantic table/column names are not required; FK topology is sufficient.

Variant D—Materials Project-flavored schema. We built a fresh 3-table PostgreSQL schema (`mp_entries`, `mp_elements`, `mp_element_ratios`) from real Materials Project REST data for 27 binary systems (299 entries), prepared 15 Japanese queries, and supplied only 8 few-shot examples written for the MP schema. No pipeline code was changed. The result was 100.0% overall (Easy 100.0% / Medium 100.0% / Hard 100.0% / Very Hard 100.0%). All 15 queries were answered correctly, confirming that the schema itself is transferable.

Together, these results show that the core design (FK introspection, schema-presented prompting, pattern-transfer few-shot) is not tied to the original 31-table schema or the L1₂ dataset. The MP transfer in particular demonstrates one-step extension to an external, real materials database using only a small schema-specific prompt and few-shot set.

Discovering hidden schema dependencies. The transfer tests also served as a verification tool: they uncovered schema-specific hard-coding (fixed references to linked table names) that had been silently accepted by the original schema. Removing these made the pipeline fully schema-independent.

10.14 Extensibility to Similar Inorganic Materials RDBs

The design principles obtained in this study are not limited to the specific database used for validation. The Steiner tree algorithm based on FK introspection operates on any normalized RDB with defined FK constraints, without requiring table additions or schema modifications. For the domain dictionary, vocabulary additions are needed when applying to new material systems (Heusler, Perovskite, etc.) or different databases, but the dictionary structure itself (4 categories: prototype aliases, element mappings, stability terms, and property terms) is applicable to inorganic materials RDBs in general.

However, the following caveats should be noted. The main evaluation of this study is on a single OQMD-derived database; while Section 10.13 demonstrated generalization to schema renaming, obfuscation, prototype expansion, and a real Materials Project-flavored schema, we do not claim generalization performance to RDBs with fundamentally different semantics, missing FK metadata, or much larger scales. Few-shot examples are dependent on the validation DB schema, and designing new examples by domain experts is essential when applying to different RDBs. With these limitations in mind, this study provides one demonstration of design principles for NL access to inorganic materials RDBs.

10.15 Limitations and Future Work

1. Single-database validation: The main evaluation of this study is on an OQMD-derived L1₂ database (31 tables, 1,559 entries). The zero-adaptation transfer tests (Section 10.13) demonstrated generalization to schema renaming (80.5%), obfuscated identifiers (90.0%), prototype expansion (95.0%), and a real Materials Project-flavored schema (97.3%), but generalization to RDBs with fundamentally different semantics or scales has not been confirmed.

2. Scope of statistical testing: Wilcoxon signed-rank tests were conducted with 5 independent runs, but the number of runs is limited; bootstrap or other methods would be useful for more precise confidence intervals.
3. Query coverage: The 100 author-designed queries were designed to cover major search patterns in materials research but do not exhaustively cover the diverse query patterns encountered in production use.
4. Gold SQL verification: Gold SQL and expected results were created by a single author; independent third-party verification has not been performed.
5. Dictionary coverage: The 497-term dictionary is specialized for $L1_2$ exploration; extension to Heusler, Perovskite, and other systems requires manual registration. Migration to semi-automatic generation from schema metadata is desirable.
6. Baseline fairness: Direct comparison with other methods (DIN-SQL, DAIL-SQL, etc.) on the same database has not been conducted.

10.16 Future Prospects

1. Validation on other inorganic materials RDBs: Verify the generality of the design principles through application to inorganic materials RDBs with different schemas, such as Materials Project and AFLOW.
2. Large-scale data validation: Validate Steiner tree search and SQL generation accuracy at the 10,000-entry scale through integration of OQMD single-element data.
3. Extended CTE evaluation: This study confirmed 100% accuracy on 5 CTE queries (Section 10.12); expand evaluation to more diverse derived quantity patterns (phase stability diagrams, elastic anisotropy metrics, etc.).
4. AI4S agent integration: Incorporate the T2SQL method as a tool for autonomous materials exploration agents via MCP protocol or function calling.
5. CALPHAD integration: A two-stage flow where Schema Graph traversal narrows candidates from the RDB, followed by CALPHAD free energy calculations for temperature-dependent phase stability assessment.
6. Multi-axis evaluation metrics: Separately evaluate SELECT column precision, JOIN path match rate, Top- k accuracy, etc., to facilitate error source identification.

11 Conclusion

This study presents design principles enabling natural language access to inorganic materials RDBs and demonstrates their effectiveness using an $L1_2$ intermetallic compound RDB as a validation target.

An ablation study of 7 conditions $\times$ 100 queries $\times$ 5 runs (3,500 evaluations total) showed that the materials domain dictionary (-7.3 pp, $p < 0.001$) and few-shot examples (-7.4 pp, $p < 0.001$) are indispensable for materials-specific query patterns. Particularly notable improvements were observed in queries involving multi-element conditions, compound property searches, and domain-specific terminology. Error analysis revealed that the remaining primary failure mode is WHERE clause condition combination errors rather than JOIN path errors (Section 7).

Materials engineering evaluation confirmed the utility as a materials exploration tool through rediscovery of 11 known $L1_2$ compounds, γ' -phase candidate ranking, and extraction of 162 stable/metastable candidates (Section 9). External validation with 100 independently designed queries achieved 72.5%; the accuracy gap from author-designed queries (roughly 12 pp) is primarily attributable to table coverage bias and few-shot optimization bias (Section 10.9).

For CTE (Common Table Expression) multi-step calculation queries (formation enthalpy calculation, etc.), the 5 patterns covered by few-shot examples achieved 100% under the full condition (dropping to 76% without few-shot examples and 86% without the dictionary), while an extended 15-query evaluation adding 10 novel patterns showed zero-shot novel patterns remaining at 32.3%, demonstrating that the scope of multi-step calculation support is governed by the few-shot pattern coverage (Section 10.12). Furthermore, a zero-adaptation transfer test on an unseen schema with all table and column names changed achieved 80.5% (Very Hard 100%), demonstrating the schema non-dependence of the design (Section 10.13).

Another important insight obtained through this study is that the normalization degree and table structure of the RDB schema are upstream factors for Text-to-SQL accuracy (Section 10.11). Schema design strategies such as complexity hiding via view definitions, self-descriptive column naming, and comprehensive FK constraint definition can contribute to accuracy improvement in parallel with pipeline-side enhancements. This perspective serves as a design guideline applicable not only to the validation DB used in this study but to inorganic materials RDBs in general with similar normalized schemas.

The Steiner tree architecture based on FK introspection can automatically follow schema changes, and with dictionary and few-shot example updates, application to other materials RDBs such as the Materials Integration (MInt) system [23] and the provenance management system pinax [24] is envisioned. While the transfer test demonstrated generalization to schema renaming, confirmation of generalization performance to RDBs with substantially different normalization structures or scales remains a future task. All evaluations are reported as means $\pm$ standard deviations from 5 independent runs, with statistical significance confirmed via Wilcoxon signed-rank tests.

Acknowledgments

The DFT calculation data for $L1_2$ intermetallic compounds are synthetic validation data constructed with reference to values from the Open Quantum Materials Database (OQMD). Attributes such as Curie temperature, $\Sigma 5$ grain boundary energy, synthesis methods, and literature references do not exist in OQMD and were generated for validation purposes. We thank the Wolverton group at Northwestern University for the development and maintenance of OQMD. This work was supported by NIMS.

Data and Code Availability

The evaluation dataset (100 gold SQL queries and expected results), ablation results (7 conditions $\times$ 100 queries in JSON), MeCab materials dictionary (497 terms, compiled .dic), Japanese reranker comparison results, and all scripts are included in the repository. Applying to other inorganic materials RDBs requires replacing the domain dictionary and few-shot examples, but the pipeline architecture and FK traversal mechanism can be reused as-is. All numerical values in the paper are output to `paper/paper_data.json` by `scripts/compute_all_figures.py`; no manually entered values are included. All 134 unit tests pass.

Author Contributions

Satoshi Minamoto: Conceptualization, methodology, software implementation, database construction, experiments, and writing.

References

- [1] J.E. Saal, S. Kirklin, M. Aykol, B. Meredig, C. Wolverton, Materials design and discovery with high-throughput density functional theory: The Open Quantum Materials Database (OQMD), *JOM* 65 (2013) 1501–1509.
- [2] S. Kirklin, J.E. Saal, B. Meredig, A. Thompson, J.W. Doak, M. Aykol, S. Rühl, C. Wolverton, The Open Quantum Materials Database (OQMD): Assessing the accuracy of DFT formation energies, *npj Comput. Mater.* 1 (2015) 15010.
- [3] A. Jain, S.P. Ong, G. Hautier, W. Chen, W.D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder, K.A. Persson, Commentary: The Materials Project: A materials genome approach to accelerating materials innovation, *APL Mater.* 1 (2013) 011002.
- [4] S. Curtarolo, W. Setyawan, G.L.W. Hart, M. Jahnatek, R.V. Chepulskii, R.H. Taylor, S. Wang, J. Xue, K. Yang, O. Levy, M.J. Mehl, H.T. Stokes, D.O. Demchenko, D. Morgan, AFLOW: An automatic framework for high-throughput materials discovery, *Comput. Mater. Sci.* 58 (2012) 218–226.
- [5] C. Draxl, M. Scheffler, NOMAD: The FAIR concept for big data-driven materials science, *MRS Bull.* 43 (2018) 676–682.
- [6] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A large-scale human-labeled dataset for complex and cross-database semantic parsing and text-to-SQL task, in: *Proc. EMNLP*, 2018, pp. 3911–3921.
- [7] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. Chang, F. Huang, R. Cheng, Y. Li, Can LLM already

- serve as a database interface? A BIG bench for large-scale database grounded text-to-SQL, in: Proc. NeurIPS, 2024.
- [8] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction, in: Proc. NeurIPS, 2024.
 - [9] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL empowered by large language models: A benchmark evaluation, in: Proc. VLDB, 2024.
 - [10] B. Wang, R. Shin, X. Liu, O. Polozov, M. Richardson, RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers, in: Proc. ACL, 2020, pp. 7567–7578.
 - [11] H. Li, J. Zhang, C. Li, H. Chen, RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL, in: Proc. AAAI, 2023.
 - [12] F. Safdarian, S. Kacupaj, SchemaGraphSQL: Schema-graph-based text-to-SQL with path finding, in: Proc. NAACL, 2026.
 - [13] T. Gupta, M. Zaki, N.M.A. Krishnan, M. Mausam, MatSciBERT: A materials domain language model for text mining and information extraction, npj Comput. Mater. 8 (2022) 102.
 - [14] N. Walker, J. Dagdelen, K. Cruse, A. Jain, A. Ceder, Extracting structured seed-mediated gold nanorod growth procedures from scientific text with MatBERT, Digital Discovery 1 (2022) 413–427.
 - [15] Y. Zhuang et al., MKNA: Materials knowledge navigation agent, Nat. Comput. Sci. (2026).
 - [16] Y. Hu et al., OptiMat: On-demand DFT-driven materials optimization, npj Comput. Mater. (2026).
 - [17] M. Ishii et al., SuperCon: Linked data platform for superconducting materials, J. Phys.: Condens. Matter (2023).
 - [18] M. Ishii et al., PoLyInfo: BFO-compliant polymer ontology, J. Chem. Inf. Model. (2024).
 - [19] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, Inf. Process. Manag. 24 (1988) 513–523.
 - [20] T. Brown et al., Language models are few-shot learners, in: Proc. NeurIPS, 2020.
 - [21] A.A. Hagberg, D.A. Schult, P.J. Swart, Exploring network structure, dynamics, and function using NetworkX, in: Proc. SciPy, 2008.

- [22] T. Mao, sqlglot: Python SQL parser and transpiler, 2023. <https://github.com/tobymao/sqlglot>
- [23] S. Minamoto, Materials integration system for process-structure-property-performance chains, Sci. Technol. Adv. Mater. (2020).
- [24] S. Minamoto, pinax: Provenance-tracking analysis workflow management system, (2026).

Supplementary Materials

Natural Language Access to Inorganic Materials
Relational Databases: Design Principles and Demonstration
via Schema-Graph-Constrained Text-to-SQL

(Japanese title: 無機材料リレーショナルデータベースの自然言語アクセス
—スキーマグラフ制約型Text-to-SQLによる設計原理と実証—)

Satoshi Minamoto

National Institute for Materials Science (NIMS),
1-2-1 Sengen, Tsukuba, Ibaraki 305-0047, Japan

`minamoto.satoshi@nims.go.jp`

This supplementary material provides detailed information on the validation and evaluation of the Schema Graph-constrained Text-to-SQL system proposed in the main paper. Section S1 presents all unit test categories and their validation focus. Section S2 lists the 39 regression test cases. Section S3 compares generated SQL examples for representative queries across three method configurations. Section S4 shows Ni_3Al neighborhood lattice constant candidates (14 representative entries out of 31). Section S5 presents the formation energy ranking of stable L_{12} candidates. Section S6 provides supplementary information on SQL injection and safety test results. Section S7 describes the LLM mode (GPT-5.5) configuration parameters and prompt structure. Section S8 lists detailed results for all 100 evaluation queries (query text, accuracy, latency). Section S9 compares queries with differing results across the 7 ablation conditions. Section S10 reports detailed results for the independently designed query pool. All values are from `paper_data.json`.

S1 Unit Test Category Details

As development safety tests, we constructed 39 regression tests across 6 categories, 6 SQL validation tests, 15 Coverage Score/parser tests, and 20 Intent Classifier tests (80 total; Table S1). Including alias resolution tests, superset detection tests, and others, all 134 unit tests pass.

S2 Regression Test Cases

Table S2 lists all 39 regression test cases (normal, no match, ambiguous, contradictory, SQL injection, and safety) with their queries and results.

Table S1: Unit test categories (all 134 tests, all PASS).

Category	ID	<i>n</i>	Validation focus
Normal	A01–A15	15	L ₁₂ elements, prototypes, stability, sorting, compound conditions
No match	B01–B05	5	Noble gas elements, non-existent combinations → 0 rows expected
Ambiguous input	C01–C10	10	Empty input, irrelevant text, single words
Contradictory	D01–D02	2	“stable and metastable,” etc.
SQL injection	E01–E05	5	DROP, DELETE, INSERT, piggyback attacks
Safety check	F01–F02	2	Direct SQL input (SELECT, UPDATE)
SQL validation	—	6	Column/table whitelist, JOIN validation
Coverage/parser	—	15	Numeric conditions, chemical formulas, Coverage Score
Intent Classifier	—	20	VASP/DFT workflow detection, classification
Alias resolution	—	15	ORDER BY/HAVING references, column aliases
Superset detection	—	10	Over-retrieval detection, row ratio thresholds
Other	—	29	Schema Graph traversal, dictionary mapping, entity extraction

S3 Generated SQL Examples

We compare generated SQL examples for representative queries across three method configurations (LLM-only, Full Schema, and our method). This complements the quantitative comparison in Table 19 by concretely showing the qualitative differences in output SQL for each configuration.

S3.1 “List stable L₁₂ compounds containing Ni”

LLM-only (no schema):

```

1 SELECT formula, stability
2 FROM materials
3 WHERE element = 'Ni'
4     AND structure_type = 'L12'
5     AND stability = 'stable';
6 -- Table hallucination: materials, stability, etc. do not exist
7 -- Not executable (ERROR: relation "materials" does not exist)

```

Full Schema (schema in prompt, no few-shot examples):

```

1 SELECT m.formula, ps.is_stable
2 FROM material_entry m
3     JOIN composition c ON c.entry_id = m.entry_id
4     JOIN structure s ON s.entry_id = m.entry_id
5     JOIN phase_stability ps ON ps.entry_id = m.entry_id
6 WHERE c.element = 'Ni'
7     AND s.strukturbericht = 'L12'
8     AND ps.is_stable = true;
9 -- Executable but incomplete:
10 -- (1) No prototype condition (misses cases where L12

```


Table S2: Regression test cases (all 39 tests, all PASS). Queries are shown in the original Japanese.

ID	Category	Natural language query (Japanese)	Result
A01	Normal	Feを含むL1 ₂ 化合物を出して	PASS
A02	Normal	安定なL1 ₂ 化合物を形成エネルギー順に出して	PASS
A03	Normal	NiとAlの両方を含む化合物を出して	PASS
A04	Normal	L1 ₂ 化合物の全リストを出して	PASS
A05	Normal	準安定なL1 ₂ 化合物を出して	PASS
A06	Normal	Coを含む安定なL1 ₂ 化合物を出して	PASS
A07	Normal	FeとAlを含むL1 ₂ 化合物を出して	PASS
A08	Normal	γ' 化合物を出して	PASS
A09	Normal	ニッケルを含むL1 ₂ 型化合物を出して	PASS
A10	Normal	L1 ₂ 化合物の全データを出して	PASS
A11	Normal	Tiを含むL1 ₂ 化合物を格子定数順（降順）に出して	PASS
A12	Normal	安定なL1 ₂ で格子定数3.5Å以上のものを出して	PASS
A13	Normal	Cu ₃ Au型化合物を出して	PASS
A14	Normal	鉄を含むL1 ₂ 化合物を出して	PASS
A15	Normal	安定なL1 ₂ 化合物でNiを含むものを出して	PASS
B01	No match	Xeを含むL1 ₂ 化合物を出して	PASS
B02	No match	UとPuを含むL1 ₂ 化合物を出して	PASS
B03	No match	RnとOgを含むL1 ₂ 化合物を出して	PASS
B04	No match	ScとIrを含む安定なL1 ₂ 化合物を出して	PASS
B05	No match	PtとGaを含む安定なL1 ₂ 化合物を出して	PASS
C01	Ambiguous	安定な化合物を出して	PASS
C02	Ambiguous	L12	PASS
C03	Ambiguous	安定なものを出して	PASS
C04	Ambiguous	(empty input)	PASS
C05	Ambiguous	今日の天気を教えて	PASS
C06	Ambiguous	Fe	PASS
C07	Ambiguous	格子定数	PASS
C08	Ambiguous	形成エネルギーが低い安定なL1 ₂ 化合物	PASS
C09	Ambiguous	NiAl L12	PASS
C10	Ambiguous	NiのL12	PASS
D01	Contradictory	安定かつ準安定なL1 ₂ 化合物	PASS
D02	Contradictory	NiなしのNi L1 ₂ 化合物	PASS
E01	Rejected	DROP TABLE material_entry;	PASS
E02	Rejected	SELECT *; DELETE FROM composition;	PASS
E03	Rejected	L1 ₂ 化合物; DROP TABLE structure;	PASS
E04	Rejected	SELECT * FROM secret_passwords	PASS
E05	Rejected	INSERT INTO material_entry VALUES (...)	PASS
F01	Safety	SELECT entry_id FROM material_entry	PASS
F02	Safety	UPDATE material_entry SET formula='X'	PASS

```

11 -- is not in strukturbericht)
12 -- (2) No DISTINCT/LIMIT
13 -- (3) Does not select materially useful columns like lattice_a

```

Our method (Schema Graph + Few-shot + Dictionary):

```

1 SELECT DISTINCT m.entry_id, m.formula,
2   s.prototype, s.lattice_a, s.space_group
3 FROM material_entry m
4   JOIN composition c ON c.entry_id = m.entry_id
5   JOIN structure s ON s.entry_id = m.entry_id
6   JOIN phase_stability ps ON ps.entry_id = m.entry_id
7 WHERE (s.prototype = 'L12'

```

```

8      OR s.strukturbericht = 'L12')
9      AND c.element = 'Ni'
10     AND ps.energy_above_hull <= 0.001
11 LIMIT 10000;
12 -- Only 3 JOINS needed via Schema Graph traversal
13 -- No table hallucination, includes DISTINCT and LIMIT

```

S3.2 “List L1₂ compounds containing both Ni and Al”

LLM-only (no schema):

```

1 SELECT * FROM compounds
2 WHERE elements LIKE '%Ni%' AND elements LIKE '%Al%'
3      AND crystal_structure = 'L12';
4 -- Table hallucination: compounds, elements column do not exist

```

Full Schema (no few-shot examples):

```

1 SELECT m.entry_id, m.formula
2 FROM material_entry m
3      JOIN composition c ON c.entry_id = m.entry_id
4      JOIN structure s ON s.entry_id = m.entry_id
5 WHERE c.element = 'Ni' AND c.element = 'Al'
6      AND s.prototype = 'L12';
7 -- Fatal: a single row cannot satisfy both conditions
8 -- simultaneously -> always 0 rows

```

Our method (EXISTS subquery):

```

1 SELECT DISTINCT m.entry_id, m.formula
2 FROM material_entry m
3      JOIN structure s ON s.entry_id = m.entry_id
4 WHERE (s.prototype = 'L12'
5        OR s.strukturbericht = 'L12')
6 AND EXISTS (
7     SELECT 1 FROM composition
8     WHERE entry_id = m.entry_id AND element = 'Ni')
9 AND EXISTS (
10     SELECT 1 FROM composition
11     WHERE entry_id = m.entry_id AND element = 'Al')
12 LIMIT 10000;
13 -- Accurate multi-element AND via EXISTS subqueries

```

S4 Ni₃Al Neighborhood Lattice Constant Candidates

Table S3 lists candidates strictly satisfying $|a - a_{\text{Ni}_3\text{Al}}| \leq 0.03 \text{ \AA}$ ($a_{\text{ref}} = 3.57 \text{ \AA}$; 14 representative entries out of 31 total).

Co₃Ti ($\Delta a = 0.02 \text{ \AA}$, $E_{\text{hull}} = 0.0$) is a stable phase and the most realistic precipitation-strengthening candidate.

Table S3: Ni₃Al neighborhood lattice constant candidates ($|a - a_{\text{ref}}| \leq 0.03 \text{ \AA}$, 14 representative entries).

Compound	$a \text{ (\AA)}$	$ \Delta a $	E_{hull}	E_f
Hf ₃ Y	3.569	0.001	0.039	-0.443
Ni ₃ Al	3.572	0.002	0.000	-0.420
Pd ₃ Mn	3.570	0.000	0.033	-0.180
Cr ₃ Al	3.567	0.003	0.082	-0.120
Cr ₃ Fe	3.574	0.004	0.037	-0.453
Rh ₃ Zr	3.574	0.004	0.026	-0.525
Cu ₃ Zr	3.575	0.005	0.038	-0.037
Ag ₃ Ti	3.564	0.006	0.039	+0.088
Au ₃ V	3.564	0.006	0.088	-0.471
Mo ₃ Nb	3.586	0.016	0.007	-0.154
Co ₃ Ti	3.550	0.020	0.000	-0.450
Pt ₃ Ge	3.550	0.020	0.021	-0.478
Ni ₃ Ge	3.583	0.013	0.075	-0.128
Ni ₃ Ga	3.598	0.028	0.030	-0.229

S5 Formation Energy Ranking of Stable L1₂ Candidates

We extracted stable/metastable L1₂ candidates satisfying $E_{\text{hull}} \leq 0.05 \text{ eV/atom}$. A total of 162 candidates were identified: 8 stable ($E_{\text{hull}} \leq 0.001$) and 154 metastable ($0.001 < E_{\text{hull}} \leq 0.05$). Among stable phases, the most negative formation energy belongs to Pt₃Al ($E_f = -0.550 \text{ eV/atom}$), followed by Al₃Sc (-0.500), Co₃Ti (-0.450), and Ni₃Al (-0.420).

S6 SQL Injection and Safety Test Details

This section provides supplementary information on the safety test results reported in Section 8. Each of SQLGuard’s 14 validation items (Section 2.8) operates independently, providing defense-in-depth against multiple threat vectors. Prohibited keyword detection is regex-based, and the table whitelist is auto-generated from schema metadata.

S7 LLM Mode Configuration

The LLM mode uses GPT-5.5 via the OpenAI API. Table S4 shows the model parameters.

The structured prompt consists of four components:

1. System message: Task description + schema YAML (table names, column names, types, FK relationships)
2. Few-shot examples: Top- k similar NL-SQL pairs selected by TF-IDF + Cross-Encoder
3. User message: Natural language query

Table S4: LLM mode configuration parameters.

Parameter	Value
Model	GPT-5.5 (OpenAI)
API access date	May–June 2026
Temperature	Not specified (GPT-5 series does not support temperature parameter)
Max tokens	4,096 (same across all methods)
Top- p	1.0
Request timeout	30 seconds
Retries	Up to 3 (exponential backoff)
n -best candidates	3
Cross-Encoder	ms-marco-MiniLM-L-6-v2
Repair loop limit	3

4. Constraints: “Generate SELECT statements only,” “Include LIMIT 10000,” “Use only tables listed in the schema”

The GPT-5 series API does not support the temperature parameter, and our implementation does not send it. Consequently, outputs may vary for identical inputs due to the LLM’s internal state. All ablation results in this paper are reported as mean $\pm$ standard deviation from 5 independent runs, with statistical significance confirmed via Wilcoxon signed-rank tests.

S8 Detailed Results for 100 Evaluation Queries

Table S5 lists all 100 evaluation queries (author-designed, E=20/M=30/H=30/VH=20) with their question text, execution accuracy (row-match recall), latency, and pass/fail ($\checkmark$ if accuracy ≥ 0.8). All values are from the ablation full condition. Queries are shown in the original Japanese.

Table S5: Detailed results for 100 evaluation queries (full condition). Queries in original Japanese.

ID	Query	Acc.	sec	P/F
Easy ($n=20$)				
q-easy_001	L1 ₂ 構造を持つ化合物を一覧にして。	1.00	29.7	$\checkmark$
q-easy_002	Niを含むL1 ₂ 型金属間化合物を抽出して。	1.00	44.3	$\checkmark$
q-easy_003	A ₃ B型組成を持つ化合物を表示して。	1.00	31.5	$\checkmark$
q-easy_004	L12型化合物の格子定数を一覧にして。	1.00	13.1	$\checkmark$
q-easy_005	Coを含む化合物を表示して。	1.00	17.2	$\checkmark$
q-easy_006	cubic構造の化合物を一覧にして。	1.00	12.6	$\checkmark$
q-easy_007	L1 ₂ 型化合物のリストを出して。	1.00	20.9	$\checkmark$
q-easy_008	Alを含む化合物を抽出して。	1.00	16.3	$\checkmark$

Continued on next page

ID	Query	Acc.	sec	P/F
q.easy_009	空間群番号221の化合物を表示して。	1.00	13.1	✓
q.easy_010	Tiを含むL1 ₂ 型化合物を一覧にして。	1.00	23.7	✓
q.easy_011	全化合物のformula一覧を出して。	1.00	12.3	✓
q.easy_012	L1 ₂ 型化合物の数を教えて。	1.00	19.9	✓
q.easy_013	Ptを含む化合物を表示して。	1.00	14.7	✓
q.easy_014	NiとAlの両方を含む化合物を抽出して。	1.00	17.9	✓
q.easy_015	Gaを含むL1 ₂ 型化合物を表示して。	1.00	37.5	✓
q.easy_016	Feを含む化合物を一覧にして。	1.00	20.1	✓
q.easy_017	Aサイトの元素を一覧にして。	1.00	10.7	✓
q.easy_018	化合物の元素系を表示して。	1.00	13.7	✓
q.easy_019	L1 ₂ 型でcubic構造の化合物を出して。	1.00	18.4	✓
q.easy_020	Wを含むL1 ₂ 型化合物を表示して。	1.00	34.4	✓
Medium (n=30)				
q.medium_001	Niを含む安定なL1 ₂ 型化合物を抽出して。	1.00	26.4	✓
q.medium_002	negative formation energyのL1 ₂ 型化合物を元素組み合わせごとに整理して。	1.00	29.0	✓
q.medium_003	Alを含む安定なL1 ₂ 型金属間化合物を抽出して。	1.00	43.3	✓
q.medium_004	Ni ₃ Alに近い格子定数を持つL1 ₂ 化合物を探して。	1.00	34.9	✓
q.medium_005	安定なL1 ₂ 型化合物を形成エネルギーが低い順に出して。	1.00	33.9	✓
q.medium_006	準安定なL1 ₂ 型化合物を抽出して。	0.84	34.3	✓
q.medium_007	L1 ₂ 型化合物の形成エネルギー分布を教えて。	1.00	24.8	✓
q.medium_008	Co含有L1 ₂ 型化合物を安定性の高い順に表示して。	1.00	25.6	✓
q.medium_009	格子定数が3.5Å以上のL1 ₂ 化合物を出して。	1.00	14.3	✓
q.medium_010	NiまたはCoを含むL1 ₂ 型化合物を一覧にして。	1.00	34.7	✓
q.medium_011	形成エネルギーが-0.4以下のL1 ₂ 型化合物を出して。	1.00	20.5	✓
q.medium_012	energy above hullが0.01以下のL1 ₂ 型化合物を表示して。	1.00	31.0	✓
q.medium_013	Ti含有L1 ₂ 型化合物の格子定数と形成エネルギーを表示して。	1.00	18.0	✓
q.medium_014	L1 ₂ 化合物の格子定数を小さい順にソートして。	1.00	16.5	✓
q.medium_015	化学系Al-Niに属するL1 ₂ 化合物を抽出して。	1.00	20.9	✓
q.medium_016	安定で格子定数が3.6Å未満のL1 ₂ 化合物を出して。	1.00	24.2	✓
q.medium_017	Nbを含むL1 ₂ 型化合物の安定性情報を出して。	1.00	30.8	✓
q.medium_018	Scを含むL1 ₂ 型化合物を格子定数と共に表示して。	1.00	26.6	✓
q.medium_019	L1 ₂ 型化合物のうちis_stableがtrueのものを出力して。	1.00	30.1	✓
q.medium_020	Ni-Al系以外のL1 ₂ 型化合物を一覧にして。	1.00	19.6	✓
q.medium_021	Aサイト元素ごとのL1 ₂ 型化合物数を集計して。	1.00	24.4	✓
q.medium_022	形成エネルギーが最も低いL1 ₂ 型化合物TOP5を出力して。	1.00	25.9	✓
q.medium_023	格子定数が4.0Å以上のL1 ₂ 化合物を出して。	1.00	16.3	✓
q.medium_024	Pdを含むL1 ₂ 型化合物を形成エネルギー順に出して。	1.00	43.6	✓

Continued on next page

ID	Query	Acc.	sec	P/F
q_medium.025	Cu含有のL1 ₂ 型化合物を安定性で分類して。	1.00	33.7	✓
q_medium.026	Bサイト元素ごとの平均形成エネルギーを出して。	0.00	29.2	—
q_medium.027	格子体積が $12\text{\AA}^3/\text{atom}$ 以下のL1 ₂ 化合物を出して。	1.00	19.8	✓
q_medium.028	Ir含有L1 ₂ 型化合物の一覧を出して。	1.00	23.7	✓
q_medium.029	2元素系のL1 ₂ 型化合物をchemical.systemで分類して。	1.00	21.9	✓
q_medium.030	安定なL1 ₂ 型化合物のうち格子定数が大きい順に並べて。	1.00	28.8	✓
Hard ($n=30$)				
q_hard.001	Ni基超合金の γ' 相候補として有望な安定/準安定L1 ₂ 化合物。	0.06	70.6	—
q_hard.002	Aサイト×Bサイト組み合わせと形成エネルギーの関係。	1.00	36.0	✓
q_hard.003	格子定数がNi ₃ Alに近く形成エネルギーが低い候補。	1.00	40.7	✓
q_hard.004	安定L1 ₂ のバルクモジュラスを高い順に表示。	1.00	29.5	✓
q_hard.005	Ni/Coを含む安定L1 ₂ を形成エネルギー順に。	1.00	45.9	✓
q_hard.006	Aサイト元素別の平均格子定数を集計。	1.00	23.5	✓
q_hard.007	$E_f \leq -0.3$ かつ $a = 3.5\text{--}3.7\text{\AA}$ のL1 ₂ 化合物。	1.00	25.7	✓
q_hard.008	バルクモジュラス180GPa以上のL1 ₂ 化合物。	1.00	26.8	✓
q_hard.009	Co基L1 ₂ でNi ₃ Alに近い格子定数、安定性順。	1.00	50.6	✓
q_hard.010	BサイトAl/TiのL1 ₂ を形成エネルギー順に。	1.00	25.4	✓
q_hard.011	L1 ₂ 化合物のせん断弾性率を表示。	1.00	23.0	✓
q_hard.012	安定で E_f 最低のL1 ₂ トップ10。	0.50	29.4	—
q_hard.013	Ni含有L1 ₂ のバルクモジュラスと E_f の関係。	1.00	38.5	✓
q_hard.014	Aサイト元素グループ別平均 E_f 。	0.00	34.8	—
q_hard.015	$a \approx 3.55\text{\AA}$ でバルクモジュラスの高いL1 ₂ 。	0.51	36.3	—
q_hard.016	格子定数と E_f の相関データ。	1.00	18.9	✓
q_hard.017	Co ₃ TiとNi ₃ Alの格子定数差を計算。	1.00	31.0	✓
q_hard.018	Al系L1 ₂ の安定/不安定のCount。	0.00	35.6	—
q_hard.019	準安定L1 ₂ をenergy above hull順にランキング。	0.84	31.0	✓
q_hard.020	Bサイト元素ごとの安定L1 ₂ 数を集計。	0.20	21.8	—
q_hard.021	DFT計算で得られた物性一覧。	1.00	36.6	✓
q_hard.022	PBE汎関数計算のバルクモジュラス。	1.00	25.3	✓
q_hard.023	Ni ₃ Alとの格子ミスマッチを小さい順に。	1.00	19.0	✓
q_hard.024	Fe含有L1 ₂ の安定性と弾性特性を同時表示。	0.00	31.2	—
q_hard.025	安定L1 ₂ でバルクモジュラス160GPa以上。	1.00	36.9	✓
q_hard.026	安定性×格子定数でNi ₃ Alに近い候補。	1.00	30.0	✓
q_hard.027	Aサイト=Ni/Co、Bサイト=Al/TiのL1 ₂ 。	1.00	34.8	✓
q_hard.028	体積あたり原子数と格子定数の関係。	1.00	23.2	✓
q_hard.029	$E_{\text{hull}} = 0$ かつ $E_f \leq -0.4$ のL1 ₂ 。	1.00	30.7	✓
q_hard.030	遷移金属含有の安定L1 ₂ 。	1.00	58.2	✓

Continued on next page

ID	Query	Acc.	sec	P/F
Very Hard ($n=20$)				
q_vhard_001	相安定性×格子ミスマッチ×弾性の多基準評価。	0.00	49.9	—
q_vhard_002	Ni ₃ Al代替/ γ' 候補の多基準探索。	0.04	48.9	—
q_vhard_003	γ' 候補の重み付きスコアでランキング。	0.00	48.3	—
q_vhard_004	Ni/Co含有×安定×高 B のトップ10。	1.00	33.5	✓
q_vhard_005	元素組み合わせ×安定性をA/Bサイト別に集計。	1.00	47.7	✓
q_vhard_006	Ni ₃ Alに近い格子×高 B ×安定の総合評価。	0.19	42.4	—
q_vhard_007	析出強化L1 ₂ の多基準（ミスマッチ× G ×安定性）。	0.05	45.8	—
q_vhard_008	Co基 γ' 候補のNi ₃ Al比較付き探索。	0.11	27.9	—
q_vhard_009	Co ₃ Tiの生成エンタルピーを純物質基準から計算（CTE）。	1.00	23.0	✓
q_vhard_010	E_f vs E_{hull} の安定性マップデータ。	1.00	48.8	✓
q_vhard_011	高温安定性の観点で E_f 上位候補。	0.09	39.2	—
q_vhard_012	Ni-Al系以外の γ' 代替候補。	0.02	37.3	—
q_vhard_013	B vs a の散布図データOutput。	1.00	18.6	✓
q_vhard_014	3d vs 4d/5d遷移金属Aサイトの比較。	0.88	53.1	✓
q_vhard_015	Ni ₃ Alより安定で B が高い候補。	0.00	49.6	—
q_vhard_016	安定L1 ₂ の ΔH_f をCTEで算出、 <-0.3 でフィルタ。	1.00	30.3	✓
q_vhard_017	安定性×格子整合×弾性の複合スコア。	1.00	68.3	✓
q_vhard_018	A-site元素ごとの平均 ΔH_f をCTEで集約。	1.00	28.2	✓
q_vhard_019	Ni含有安定L1 ₂ の ΔH_f を多段CTEで算出、Top5。	1.00	32.7	✓
q_vhard_020	形成 E が加重純物質基準より0.1以上低い化合物（CTE）。	1.00	23.9	✓

S9 Accuracy Difference Queries Between Ablation Conditions

Table S6 shows 35 queries with accuracy differences across the 7 ablation conditions. Bold values indicate > 0.05 decrease from the full condition. –FS = no_fewshot, –Dict = no_dict, –RR = no_reranker, –GD = no_guard, –NB = no_nbest, –GR = no_graph.

From Table S6, the following trends are observed:

- no_fewshot (–FS) causes widespread zeroing across Easy–Hard tiers, demonstrating the broad contribution of few-shot examples.
- no_dict (–Dict) concentrates zeroing in the Very Hard tier, confirming the domain dictionary’s indispensability for VH queries.
- no_reranker (–RR) causes complete failure on q_hard_003, q_hard_026, q_medium_004, and q_medium_020, demonstrating the effect of candidate selection.
- no_guard (–GD) zeroes all CTE queries (q_vhard_009/016/018/019/020), demonstrating that SQLGuard is essential for CTE syntax safety verification.
- q_vhard_007 scores 0.05 under the full condition but rises to 1.00 under –RR/–GD/–NB/–GR—an example where the reranker and SQLGuard over-filter.

Table S6: Queries with accuracy differences across ablation conditions (35/100 queries). Bold indicates > 0.05 decrease from full condition.

ID	D	Full	−FS	−Dict	−RR	−GD	−NB	−GR
q_easy_007	E	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_medium_001	M	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_medium_004	M	1.00	1.00	1.00	0.00	1.00	1.00	1.00
q_medium_012	M	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_medium_013	M	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_medium_014	M	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_medium_020	M	1.00	1.00	1.00	0.00	1.00	1.00	1.00
q_medium_021	M	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_medium_026	M	0.00	0.05	0.00	0.00	0.00	0.00	0.00
q_hard_001	H	0.06	0.00	0.00	0.06	0.06	0.06	0.06
q_hard_002	H	1.00	1.00	0.00	1.00	1.00	1.00	1.00
q_hard_003	H	1.00	0.00	1.00	0.01	1.00	1.00	1.00
q_hard_004	H	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_hard_006	H	1.00	1.00	0.00	1.00	1.00	1.00	1.00
q_hard_010	H	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_hard_011	H	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_hard_012	H	0.50	0.50	0.38	0.50	0.50	0.50	0.50
q_hard_024	H	0.00	0.00	0.00	0.11	0.00	0.00	0.00
q_hard_026	H	1.00	1.00	1.00	0.01	1.00	1.00	1.00
q_vhard_001	V	0.00	0.00	0.00	0.19	0.00	0.00	0.00
q_vhard_002	V	0.04	0.17	0.19	0.03	0.02	0.00	0.03
q_vhard_004	V	1.00	1.00	0.00	1.00	1.00	1.00	1.00
q_vhard_005	V	1.00	1.00	0.00	1.00	1.00	0.00	1.00
q_vhard_006	V	0.19	0.19	0.00	0.19	0.19	0.19	0.19
q_vhard_007	V	0.05	0.00	0.00	1.00	1.00	1.00	1.00
q_vhard_008	V	0.11	0.11	0.00	0.11	0.11	0.11	0.11
q_vhard_009	V	1.00	0.00	1.00	1.00	0.00	1.00	1.00
q_vhard_012	V	0.02	0.18	0.00	0.18	0.18	0.18	0.18
q_vhard_013	V	1.00	0.00	1.00	1.00	1.00	1.00	1.00
q_vhard_015	V	0.00	0.02	0.00	0.02	0.02	0.00	0.02
q_vhard_016	V	1.00	0.00	1.00	1.00	0.00	1.00	1.00
q_vhard_017	V	1.00	0.60	0.00	1.00	1.00	1.00	1.00
q_vhard_018	V	1.00	0.00	1.00	1.00	0.00	1.00	1.00
q_vhard_019	V	1.00	0.00	0.60	1.00	0.00	1.00	1.00
q_vhard_020	V	1.00	0.00	1.00	1.00	0.00	1.00	1.00

S10 Detailed Results for Independent Query Pool

Table S7 shows the difficulty-stratified accuracy for the independently designed queries reported in Section 10.9, drawn from the original 100-query pool (categories A–J). Of these, 48 queries were selected into the 60-query harmonized set based on the unified classification criteria (12 Very Hard queries were newly designed). Difficulty levels are E (Easy), M (Medium), H (Hard), and VH (Very Hard); a query is counted as correct if execution F1 ≥ 0.8 .

The Very Hard accuracy of 16.7% (requiring 5–7 table JOINS) is even lower than author-designed Very Hard (32.7%), indicating that JOIN path discovery to uncovered tables is a primary challenge.

The independent query pool comprises 10 categories (A: basic search, B: composition/elements, C: structure/lattice constants, D: stability/formation energy, E: DFT calculations/properties, F: electronic structure/magnetic/thermal properties,

Table S7: Independently designed query accuracy by difficulty (unified complexity score criterion).

Difficulty	Count	Correct	Acc. rate	Mean exec. acc.
Easy (< 8)	12	9	75.0%	83.3%
Medium (8–11)	18	10	55.6%	69.9%
Hard (12–16)	18	11	61.1%	70.1%
Very Hard (≥ 17)	12	2	16.7%	19.4%
Overall	60	32	53.3%	62.5%

G: surface/grain boundary/defects, H: literature/synthesis/applications, I: materials design/screening, J: comparison/statistics). Of 100 total queries (10 per category), 48 were selected for the harmonized set, combined with 12 newly designed VH queries to form the 60-query evaluation set.